Artificial Intelligence

Planning & Reinforcement Learning

Lecture Notes

Master's Program in Artificial Intelligence

ACADEMIC YEAR

2025–2026

Jacopo Parretti

Contents

Part I

Classical Planning

1 Introduction to Planning

1.1 Historical Context and Motivation

Planning has been a central topic in artificial intelligence since the inception of the field at the **Dartmouth Conference in 1956**, where the founding fathers of AI first gathered to define the scope and ambitions of this new discipline. Planning represents one of the fundamental capabilities required for intelligent behavior: the ability to reason about sequences of actions that achieve desired goals.

The motivations for studying automated planning are manifold:

- **Autonomous agents**: Enabling robots, software agents, and autonomous systems to make decisions and act independently in complex environments
- **Resource optimization**: Finding optimal or near-optimal strategies for resource allocation, scheduling, and logistics
- **Scientific applications**: Modeling and solving problems in domains such as space exploration, manufacturing, and healthcare
- **Theoretical foundations**: Understanding the computational complexity and formal properties of reasoning about action and change

1.2 The Planning Cycle

The complete planning process involves several interconnected phases that form a continuous cycle:

1. **Plan Generation**: Given a description of the current state, available actions, and desired goals, synthesize a sequence of actions (a plan) that transforms the initial state into a goal state. This phase involves:
    - Analyzing the initial state to understand what is currently true
    - Identifying the gap between the current state and the goal state
    - Searching through the space of possible action sequences
    - Evaluating candidate plans for correctness and optimality
    - Selecting the best plan according to specified criteria (e.g., shortest length, minimum cost, fastest execution)
2. **Plan Deployment**: Execute the generated plan in the actual environment, monitoring the execution to detect deviations from expected behavior. This phase includes:
    - Translating abstract plan actions into concrete executable commands
    - Monitoring sensors and feedback to verify that actions have the expected effects
    - Detecting discrepancies between predicted and actual states
    - Maintaining a record of executed actions for debugging and learning

3. **Replanning**: When execution monitoring detects that the current plan is no longer valid (due to unexpected events, action failures, or environmental changes), generate a new plan or repair the existing one. Replanning strategies include:
    - **Plan repair**: Modify the existing plan minimally to accommodate the new situation
    - **Replan from scratch**: Generate an entirely new plan from the current state
    - **Contingency planning**: Use pre-computed alternative plans for anticipated failures

This cycle reflects the reality that planning systems must operate in dynamic, partially unpredictable environments where initial plans may need adaptation. The cycle continues iteratively until the goal is achieved or deemed unreachable.

1.2.1 Challenges in Real-World Planning

Real-world deployment of planning systems faces several challenges:

- **Execution uncertainty**: Actions may fail or have unexpected outcomes
- **Incomplete information**: The planner may not have complete knowledge of the environment
- **Dynamic environments**: The world may change while the plan is being executed
- **Computational constraints**: Planning must often occur in real-time with limited computational resources
- **Plan quality vs. planning time**: Trade-off between finding optimal plans and responding quickly

These challenges motivate the development of robust planning algorithms that can handle uncertainty, adapt to changes, and operate efficiently under resource constraints.

2 Formalization of the Planning Problem

2.1 State Transition Systems

The mathematical foundation of planning rests on the concept of a **state transition system**, which provides a formal model of how actions transform states.

Definition 2.1 (State Transition System)**.** A state transition system is a tuple $\Sigma = \langle S, A, \gamma \rangle$ where:

- S is a finite or countably infinite set of **states**
- A is a finite or countably infinite set of **actions**
- $\gamma : S \times A \to S$ is a **state transition function** that maps a state and an action to a resulting state

The transition function $\gamma(s, a) = s'$ specifies that executing action a in state s results in state s'. This function encodes the **dynamics** of the domain—how the world changes in response to actions.

2.1.1 Properties of State Transition Systems

State transition systems can be characterized by several important properties:

- **Determinism**: In a deterministic system, γ is a function—each state-action pair leads to exactly one successor state. In non-deterministic systems, multiple outcomes are possible.
- **Reachability**: A state s' is reachable from state s if there exists a sequence of actions that transforms s into s'. The reachable state space from an initial state is often much smaller than the total state space.
- **Reversibility**: An action is reversible if there exists another action that undoes its effects. Many real-world actions are irreversible (e.g., breaking an object).
- **State space structure**: The connectivity and topology of the state space significantly affect planning complexity. Highly connected spaces may have many solution paths, while sparse spaces may have few or no solutions.

2.2 Classical Planning: Fundamental Assumptions

Classical planning makes several simplifying assumptions that restrict the class of problems considered but enable efficient algorithmic solutions.

Classical Planning Framework

The classical planning framework relies on six key assumptions:

1. Finitely many states
2. Finitely many actions
3. Deterministic transition function
4. Full observability
5. Instantaneous actions
6. No exogenous events

1. **Finitely many states**: The state space S is finite, allowing exhaustive search techniques. This assumption ensures that the planning problem is decidable and that search algorithms will terminate.

Justification: While real-world domains may have infinite state spaces (e.g., continuous variables), finite approximations are often sufficient for practical purposes.

2. **Finitely many actions**: The action space A is finite, ensuring decidability. Each state has a finite branching factor in the state space graph.

   Justification: Even in complex domains, the number of distinct action types is typically manageable, though the number of ground actions (instantiated with specific objects) may be large.

3. **Deterministic transition function**: For every state s and action a, there is exactly one resulting state $\gamma(s, a)$. Actions have predictable, certain outcomes.

   Justification: Determinism simplifies reasoning about action effects. Non-deterministic planning requires more complex formalisms (e.g., Markov Decision Processes).

4. **Full observability**: The planner has complete knowledge of the current state at all times. There is no uncertainty about which state the system occupies.

   Justification: Full observability eliminates the need for belief state reasoning and sensing actions. Partial observability requires more sophisticated planning approaches.

5. **Instantaneous actions**: Actions have no duration; they occur instantaneously. Temporal reasoning is not required.

   Justification: Ignoring action durations simplifies the planning model. Temporal planning extends classical planning to handle durations and concurrent actions.

6. **No exogenous events**: Only the planning agent can change the world through deliberate actions. The environment remains static unless acted upon.

   Justification: Exogenous events (e.g., other agents, natural processes) introduce additional complexity. Classical planning assumes a static world between actions.

These assumptions, while restrictive, capture a significant and important class of planning problems and provide a foundation for understanding more complex, realistic planning scenarios.

2.2.1 Relaxing Classical Assumptions

Modern planning research has developed extensions that relax these assumptions:

- **Probabilistic planning**: Handles non-deterministic actions with probability distributions over outcomes
- **Conformant planning**: Plans under partial observability without sensing
- **Contingent planning**: Generates conditional plans that include sensing actions
- **Temporal planning**: Reasons about action durations and temporal constraints
- **Multi-agent planning**: Coordinates plans among multiple agents

2.3 The Planning Problem

Given a state transition system $\Sigma = \langle S, A, \gamma \rangle$, we can now formally define the planning problem.

Planning Problem

Definition 2.2 (Planning Problem)**.** A planning problem is a tuple $P = \langle \Sigma, s_0, G \rangle$ where:

- $\Sigma = \langle S, A, \gamma \rangle$ is a state transition system
- $s_0 \in S$ is the **initial state**
- $G \subseteq S$ is a set of **goal states**

Alternatively, goals can be specified as a **goal formula** g, a logical expression that is satisfied by exactly those states in G. This allows more compact and expressive goal specifications.

2.3.1 Goal Specification

Goals can be specified in several ways:

- **Explicit goal states**: Enumerate the set G of acceptable final states. This is impractical for large state spaces.
- **Goal formula**: A logical formula g such that $G = \{s \in S \mid s \models g\}$. For example, in propositional logic: $g = \texttt{At}(\text{Robot}, \text{RoomB}) \wedge \texttt{Clean}(\text{RoomB})$
- **Goal conditions**: A set of propositions that must be true in the goal state. This is the most common approach in classical planning.
- **Utility functions**: In optimization settings, goals may be specified as utility functions to maximize or cost functions to minimize.

Solution Plan

Definition 2.3 (Solution Plan)**.** A **solution** to a planning problem $P = \langle \Sigma, s_0, G \rangle$ is a sequence of actions $\pi = \langle a_1, a_2, \ldots, a_n \rangle$ such that:

$$s_n = \gamma(\gamma(\cdots \gamma(\gamma(s_0, a_1), a_2) \cdots, a_{n-1}), a_n) \in G$$

That is, executing the sequence of actions starting from the initial state s_0 results in a goal state.

We can also write this more compactly using the notation $\gamma^*(s, \pi)$ to denote the state reached by executing plan π from state s:

$$\gamma^*(s_0, \pi) \in G$$

2.3.2 Plan Quality

Not all solution plans are equally desirable. Common quality metrics include:

- **Plan length**: Number of actions in the plan. Shorter plans are often preferred.
- **Plan cost**: Sum of action costs. Each action a may have an associated cost $c(a)$.
- **Makespan**: Total execution time (relevant in temporal planning).
- **Resource consumption**: Amount of resources used during execution.

An **optimal plan** minimizes the chosen quality metric among all solution plans.

3 Representation of Planning Problems

3.1 The Challenge of Explicit Representation

While the state transition system formalism is mathematically elegant, it faces a critical practical challenge: **the curse of dimensionality**. Real-world planning domains can have exponentially large state spaces. For example:

- A domain with n boolean variables has 2^n possible states
- A domain with k objects and m binary relations has up to 2^{mk^2} states
- Enumerating all states and transitions explicitly is infeasible for even moderately-sized problems

Example 3.1 (Blocks World Complexity)**.** Consider a Blocks World domain with n blocks. The number of possible configurations grows super-exponentially with n. For $n = 10$ blocks, there are more than 10^{13} possible configurations. Explicitly representing the transition function would require storing information about trillions of state-action pairs.

This motivates the need for **compact, structured representations** that exploit regularities in the domain to represent large state spaces and transition functions implicitly.

3.2 Action Schemas: Structured Representation

The classical approach to compact representation uses **action schemas** (also called action templates or operators). An action schema is a parameterized description of a family of related actions.

Action Schema

Definition 3.1 (Action Schema)**.** An action schema consists of:
- **Name and parameters**: A symbolic name and typed parameters, e.g., $\texttt{Move}(x, y)$
- **Preconditions**: A logical formula Pre specifying when the action can be executed
- **Effects**: A logical formula Eff specifying how the state changes when the action is executed

Actions are **ground instances** of action schemas, obtained by binding the parameters to specific objects in the domain. For example, the schema $\texttt{Move}(x, y)$ might generate ground actions $\texttt{Move}(\text{RoomA}, \text{RoomB})$, $\texttt{Move}(\text{RoomB}, \text{RoomC})$, etc.

3.2.1 Advantages of Action Schemas

Action schemas provide several benefits:

- **Compactness**: A single schema can represent exponentially many ground actions
- **Generality**: Schemas capture the general structure of actions independent of specific objects
- **Scalability**: Adding new objects to the domain automatically generates new applicable actions
- **Knowledge reuse**: Schemas learned in one domain can be transferred to similar domains

3.3 State Representation: Propositional Logic

In classical planning, states are typically represented using **propositional logic**:

- A state is a set of **atoms** (propositional variables) that are true in that state
- Atoms not in the set are assumed false (**closed-world assumption**)
- Action preconditions and effects are logical formulas over these atoms

This representation allows:

- Compact encoding of structured states
- Efficient reasoning about action applicability and effects
- Use of logical inference techniques for plan generation

3.3.1 State Update Semantics

STRIPS Assumption

When an action is executed, the state is updated according to the action's effects:

- **Add list**: Atoms that become true after the action
- **Delete list**: Atoms that become false after the action
- **Frame axioms**: Atoms not mentioned in the effects remain unchanged

The **STRIPS assumption** (named after the Stanford Research Institute Problem Solver) states that only atoms explicitly mentioned in the effects change; all other atoms persist unchanged. This simplifies reasoning about action effects.

3.4 Example: Blocks World

Consider the classic **Blocks World** domain, one of the most studied domains in planning research.

3.4.1 Domain Description

The Blocks World consists of:

- A set of blocks that can be stacked on top of each other
- A table with unlimited space
- A robot arm that can pick up and put down blocks

Blocks World State Representation

State predicates:
- $\texttt{On}(x, y)$: Block x is directly on top of block y
- $\texttt{OnTable}(x)$: Block x is on the table
- $\texttt{Clear}(x)$: Block x has no blocks on top of it
- $\texttt{Holding}(x)$: The robot arm is holding block x
- $\texttt{ArmEmpty}$: The robot arm is not holding anything

3.4.2 Action Schemas

PickUp(x): Pick up block x from the table

Parameters: x (block)
Precondition: $\texttt{Clear}(x) \land \texttt{OnTable}(x) \land \texttt{ArmEmpty}$
Effect: $\texttt{Holding}(x) \land \neg\texttt{OnTable}(x) \land \neg\texttt{Clear}(x) \land \neg\texttt{ArmEmpty}$

PutDown(x): Put down block x on the table

Parameters: x (block)
Precondition: $\texttt{Holding}(x)$
Effect: $\texttt{OnTable}(x) \land \texttt{Clear}(x) \land \texttt{ArmEmpty} \land \neg\texttt{Holding}(x)$

Stack(x, y): Stack block x on top of block y

Parameters: x, y (blocks)
Precondition: $\texttt{Holding}(x) \land \texttt{Clear}(y)$
Effect: $\texttt{On}(x, y) \land \texttt{Clear}(x) \land \texttt{ArmEmpty} \land \neg\texttt{Holding}(x) \land \neg\texttt{Clear}(y)$

Unstack(x, y): Remove block x from on top of block y

Parameters: x, y (blocks)
Precondition: $\texttt{On}(x, y) \land \texttt{Clear}(x) \land \texttt{ArmEmpty}$
Effect: $\texttt{Holding}(x) \land \texttt{Clear}(y) \land \neg\texttt{On}(x, y) \land \neg\texttt{Clear}(x) \land \neg\texttt{ArmEmpty}$

3.4.3 Example Problem Instance

Initial state: Blocks A, B, C are all on the table

$$s_0 = \{\texttt{OnTable}(A), \texttt{OnTable}(B), \texttt{OnTable}(C), \texttt{Clear}(A), \texttt{Clear}(B), \texttt{Clear}(C), \texttt{ArmEmpty}\}$$

Goal: Stack the blocks in the order A on B on C

$$G = \{\texttt{On}(A, B), \texttt{On}(B, C), \texttt{OnTable}(C)\}$$

Solution plan:

1. $\texttt{PickUp}(B)$
2. $\texttt{Stack}(B, C)$
3. $\texttt{PickUp}(A)$
4. $\texttt{Stack}(A, B)$

This plan has length 4 and achieves the goal from the initial state.

Part II

Advanced Planning Representations

4 Reduction to Propositional Satisfiability

4.1 Motivation

We continue our discussion of the formalization and representation of planning problems. A fundamental technique in automated planning is to **reduce the planning problem to the satisfiability problem of propositional logic**. This reduction allows us to leverage powerful SAT solvers to find plans efficiently.

Reduction to SAT

The key idea is to encode:

- The initial state as a propositional formula
- The goal conditions as a propositional formula
- The action effects and preconditions as propositional formulas
- The constraint that actions must form a valid sequence

If the resulting formula is satisfiable, the satisfying assignment corresponds to a valid plan.

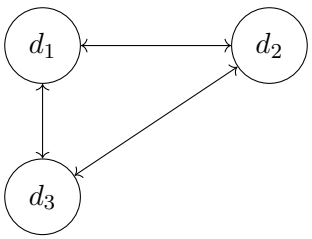
4.2 Example Domain: Dock Worker Robots (DWR)

To illustrate these concepts, we introduce the **Dock Worker Robots (DWR)** domain, a classic planning benchmark that models container logistics at a port.

4.2.1 Domain Description

The DWR world consists of:

- **Docks**: Three docks d_1, d_2, d_3 arranged in a triangular configuration
- **Connectivity**: Docks are connected as follows: $d_1 \leftrightarrow d_3$, $d_1 \leftrightarrow d_2$, $d_2 \leftrightarrow d_3$
- **Piles**: Storage locations at docks where containers can be stacked
- **Containers**: Objects that need to be moved between locations
- **Robots**: Mobile agents that can move containers between piles



4.2.2 Initial State Configuration

The initial state of our example problem is:

- **At dock** d_1: Pile p_1 contains two containers: c_1 (on top) and c_2 (below)
- **At dock** d_2:

  - Pile p_2 contains one container: c_3
  - Pile p_3 is empty
- **At dock** d_3: Pile p_4 is empty

4.3 Formal Representation

4.3.1 Objects and Types

The domain contains objects of different **sorts** (types). Each object is represented by a constant symbol with an associated type:

Docks:

$$\text{Docks} = \{d_1, d_2, d_3\}$$

where each d_i is a constant symbol of sort `Dock`.

Containers:

$$\text{Containers} = \{c_1, c_2, c_3\}$$

where each c_i is a constant symbol of sort `Container`.

Robots:

$$\text{Robots} = \{r_1, r_2\}$$

where each r_i is a constant symbol of sort `Robot`.

Piles:

$$\text{Piles} = \{p_1, p_2, p_3, p_4\}$$

where each p_i is a constant symbol of sort `Pile`.

The complete set of objects is:

$$\text{Objects} = \text{Docks} \cup \text{Piles} \cup \text{Containers} \cup \text{Robots}$$

4.3.2 Static Relations

Some relations in the domain are **static**—they do not change during plan execution. These can be specified once and remain constant throughout.

Adjacency relation: Specifies which docks are directly connected, allowing robots to travel between them.

$$\texttt{adjacent} = \{(d_1, d_3), (d_3, d_1), (d_2, d_3), (d_3, d_2), (d_1, d_2), (d_2, d_1)\}$$

This relation is symmetric: if dock d_i is adjacent to dock d_j, then d_j is adjacent to d_i.

Pile-Dock association: Each pile is permanently located at a specific dock.

$$\begin{gathered}\texttt{at}(p_1, d_1)\\ \texttt{at}(p_2, d_2)\\ \texttt{at}(p_3, d_2)\\ \texttt{at}(p_4, d_3)\end{gathered}$$

4.3.3 Rigid Relations

The adjacency relation is defined over $\texttt{Docks} \times \texttt{Docks}$. This is called a **rigid relation** (also known as a **static relation**)—it does not change during plan execution. Actions cannot modify the physical layout of the docks or their connectivity.

Rigid relations are important because:

- They reduce the state space by factoring out unchanging aspects
- They can be checked once at planning time rather than during execution
- They simplify action preconditions by providing fixed background knowledge

4.4 State Representation in DWR

4.4.1 What is a State?

A **state** is an assignment of values to propositional variables. More precisely, a state can be represented in two equivalent ways:

1. **First-order ground atoms**: Predicates where all arguments are constants (no variables)
2. **First-order ground terms**: Terms where all arguments are constants

Important restriction: We only allow **flat expressions**, meaning expressions with depth 1 (no nesting).

Example 4.1 (Forbidden Nesting)**.** Consider the predicate $\texttt{loc}(x, y)$ meaning "the location of x is y". We do **not** allow nested expressions such as:

$$\texttt{loc}(r_1, \texttt{loc}(r_2))$$

This would represent "the location of r_1 is the location of r_2", which violates the flatness constraint.

All atoms and terms must be flat—arguments of predicates and functions must be constants, not complex expressions.

4.4.2 State as Truth Assignment

A state assigns truth values to ground atoms. Consider describing the initial state s_0 of our DWR example:

$$\begin{gathered}\texttt{loc}(r_1, d_1) \\ \texttt{pos}(p_1, d_1) \\ \texttt{pos}(c_2, p_1)\end{gathered}$$

These atoms being listed in the state means they are **implicitly assigned true**:

$$\begin{aligned}\texttt{loc}(r_1, d_1) &\leftarrow \text{true} \\ \texttt{pos}(p_1, d_1) &\leftarrow \text{true} \\ \texttt{pos}(c_2, p_1) &\leftarrow \text{true}\end{aligned}$$

Atoms not mentioned in the state are assumed false (closed-world assumption).

4.5 Two Representation Formalisms

There are two common approaches to representing planning problems, differing in how they encode state information.

4.5.1 Classical (Propositional) Representation

In the **classical representation**, states are represented using only **Boolean assignments** to propositional atoms. Each possible fact about the world is represented as a Boolean variable.

Example atoms:

- $\texttt{loc}(r_1, d_1)$: Robot r_1 is at dock d_1 (true/false)
- $\texttt{pos}(c_2, p_1)$: Container c_2 is at pile p_1 (true/false)
- $\texttt{empty}(r_1)$: Robot r_1 is not carrying anything (true/false)

Characteristics:

- Simple and uniform representation
- Direct correspondence to propositional logic
- Easy to encode as SAT problems
- May require many atoms for complex domains

4.5.2 State Variable Representation

The **state variable representation** uses a slightly different syntax based on **function symbols**. Instead of Boolean atoms, we use variables that can take on different values from a finite domain.

Example:

$$\texttt{loc}(r_1) = d_1$$

This means "the location of robot r_1 is d_1". Here, $\texttt{loc}(r_1)$ is a **state variable** (a function that returns a value), and d_1 is its current value.

Comparison with classical representation:

Classical	State Variable
$\texttt{loc}(r_1, d_1) = \text{true}$ $\texttt{loc}(r_1, d_2) = \text{false}$ $\texttt{loc}(r_1, d_3) = \text{false}$	$\texttt{loc}(r_1) = d_1$

Advantages of state variables:

- More compact: one variable instead of multiple Boolean atoms
- Explicitly represents mutual exclusion (robot can only be at one location)
- Natural for domains with multi-valued attributes
- Reduces the number of variables in SAT encodings

Relationship: The two representations are **equivalent in expressive power**. Any problem representable in one formalism can be translated to the other. The choice depends on:

- The planning algorithm being used
- The structure of the domain

- Efficiency considerations for the specific problem

4.5.3 Example: Complete State Representation

Let us represent the complete initial state s_0 of our DWR example in both formalisms.

Classical representation (partial):

$$\begin{aligned}
&\texttt{loc}(r_1, d_1) = \text{true}\\
&\texttt{loc}(r_2, d_2) = \text{true}\\
&\texttt{pos}(c_1, p_1) = \text{true}\\
&\texttt{pos}(c_2, p_1) = \text{true}\\
&\texttt{pos}(c_3, p_2) = \text{true}\\
&\texttt{on}(c_1, c_2) = \text{true}\\
&\texttt{empty}(r_1) = \text{true}\\
&\texttt{empty}(r_2) = \text{true}\\
&\texttt{top}(c_1, p_1) = \text{true}\\
&\texttt{top}(c_3, p_2) = \text{true}
\end{aligned}$$

State variable representation:

$$\begin{aligned}
&\texttt{loc}(r_1) = d_1\\
&\texttt{loc}(r_2) = d_2\\
&\texttt{pos}(c_1) = p_1\\
&\texttt{pos}(c_2) = p_1\\
&\texttt{pos}(c_3) = p_2\\
&\texttt{on}(c_1) = c_2\\
&\texttt{loaded}(r_1) = \text{nil}\\
&\texttt{loaded}(r_2) = \text{nil}\\
&\texttt{top}(p_1) = c_1\\
&\texttt{top}(p_2) = c_3
\end{aligned}$$

Both representations capture the same information but organize it differently. The state variable representation is more compact and explicitly enforces constraints (e.g., a robot can only be at one location).

4.6 Key Insight: States as Boolean Assignments

The fundamental insight is that regardless of the representation formalism chosen:

A state is fundamentally a Boolean assignment

Whether we use propositional atoms or state variables, we are ultimately assigning truth values (or equivalently, values from finite domains) to a set of variables.

5 Action Schemas in DWR

5.1 Structure of Action Schemas

An **action schema** α is a template for generating ground actions. It consists of:

Action Schema Components

Definition 5.1 (Action Schema)**.** An action schema α has the following components:

- **Name and parameters**: $\alpha : \texttt{name}(z_1, z_2, \ldots, z_k)$ where z_i are typed parameters
- **Preconditions**: $\text{Pre}(\alpha) = \{p_1, p_2, \ldots, p_n\}$ where each p_i is a literal
- **Effects**: $\text{Eff}(\alpha) = \{q_1, q_2, \ldots, q_m\}$ where each q_i is a literal
- **Cost**: $c(\alpha) \in \mathbb{N}_0$ (typically a non-negative integer)

Important constraint: The parameters z_i of α can appear in the precondition literals p_j and effect literals q_j. The arguments in these literals can be either:

- **Constants**: Specific objects from the domain
- **Parameters**: Variables that will be instantiated

No arbitrary free variables are allowed—all variables must be parameters of the action schema.

Ground Action

Definition 5.2 (Ground Action)**.** A **ground action** a is obtained by substituting all parameters z_i in action schema α with specific constants from the domain. The action a is an **instance** of α.

5.2 Example: Take/Load Action

Consider the action of a robot taking a container from a pile. We define the action schema:

$$\alpha : \texttt{take}(r, c, c', p, d)$$

Parameters:

- r: robot (type `Robot`)
- c: container to take (type `Container`)
- c': container below c (type `Container` $\cup$ {`nil`})
- p: pile (type `Pile`)
- d: dock (type `Dock`)

5.2.1 Correcting Static Relations

Before defining the preconditions, we note that there are **two rigid (static) relations** in the DWR domain:

1. `adjacent` : `Docks` $\times$ `Docks` — specifies which docks are connected
2. `at` : `Piles` $\times$ `Docks` — specifies which pile is located at which dock

Both relations are rigid because the physical layout of the port does not change during plan execution.

5.2.2 Preconditions

For the `take` action to be executable, the following conditions must hold:

$$\text{Pre}(\texttt{take}(r, c, c', p, d)) = \left\{ \begin{array}{l} \texttt{loc}(r) = d \\ \texttt{at}(p, d) \\ \texttt{pos}(c) = p \\ \texttt{on}(c) = c' \\ \texttt{top}(p) = c \\ \texttt{cargo}(r) = \texttt{nil} \end{array} \right\}$$

Interpretation:

- The robot r is at dock d
- The pile p is located at dock d (static relation)
- Container c is positioned at pile p
- Container c is on top of container c' (or c' is `nil` if c is at the bottom)
- Container c is the top container of pile p
- The robot r is not carrying anything

5.2.3 Effects

When the `take` action is executed, the following changes occur:

$$\text{Eff}(\texttt{take}(r, c, c', p, d)) = \left\{ \begin{array}{l} \texttt{cargo}(r) = c \\ \texttt{top}(p) = c' \\ \texttt{pos}(c) = r \end{array} \right\}$$

Interpretation:

- The robot r is now carrying container c
- The top of pile p is now c' (the container that was below c)
- The position of container c is now the robot r

5.3 Action Applicability

Action Applicability

Definition 5.3 (Action Applicability)**.** A ground action a is **applicable** (or **executable**) in state s if all preconditions of a are satisfied in s:

$$a \text{ applicable in } s \quad \Leftrightarrow \quad s \models \text{Pre}(a)$$

5.3.1 Examples

Consider two ground actions:

Example 1: $a_1 = \texttt{take}(r_1, c_1, c_2, p_1, d_1)$

In the initial state s_0 where:

- $\texttt{loc}(r_1) = d_1$
- $\texttt{at}(p_1, d_1)$

- $\texttt{pos}(c_1) = p_1$
- $\texttt{on}(c_1) = c_2$
- $\texttt{top}(p_1) = c_1$
- $\texttt{cargo}(r_1) = \texttt{nil}$

All preconditions are satisfied, so a_1 is **executable/applicable** in s_0.

Example 2: $a_2 = \texttt{take}(r_2, c_3, \texttt{nil}, p_4, d_3)$

This action requires:

- $\texttt{loc}(r_2) = d_3$
- $\texttt{at}(p_4, d_3)$
- $\texttt{pos}(c_3) = p_4$
- $\texttt{top}(p_4) = c_3$

However, in s_0, we have $\texttt{pos}(c_3) = p_2$ (not p_4) and $\texttt{loc}(r_2) = d_2$ (not d_3). Therefore, a_2 is **not executable/not applicable** in s_0.

6 State Transition Function

6.1 Formal Definition

The **state transition function** γ defines how actions transform states:

$$\gamma : \text{State} \times \text{Action} \to \text{State}$$

where $\gamma(s, a)$ computes the successor state obtained by executing ground action a in state s.

6.2 Computing the Successor State

Given a state s and a ground action a (which is a ground instance of an action schema α), the successor state $\gamma(s, a) = s'$ is defined if and only if:

1. $s \models p$ for all $p \in \text{Pre}(a)$ (all preconditions are satisfied)
2. s' is consistent (the resulting state contains no contradictions)

The successor state s' is computed as:

$$s' = \{q \mid q \in \text{Eff}(a)\} \cup \{q \mid s \models q, \neg q \notin \text{Eff}(a), q \notin \text{Eff}(a)\}$$

Interpretation: The new state consists of:

- All literals in the effects of a
- All literals from s that are not affected by a (neither the literal nor its negation appears in the effects)

6.3 Goal States

While the initial state is a specific assignment, goals are typically specified more flexibly.

Goal Specification

Definition 6.1 (Goal Formula)**.** A **goal formula** G is a conjunction of literals specifying desired properties of goal states.

Definition 6.2 (Goal States)**.** The set of **goal states** is:

$$S_g = \{s \mid s \models G\}$$

That is, all state assignments that satisfy the goal formula G.

6.4 Comparing Representations: Simplified DWR Example

Let us compare the classical and state variable representations using a simplified version of the DWR domain with three action schemas.

6.4.1 Classical Representation

In the classical representation, we use propositional atoms to represent facts:

Action 1: $\texttt{take}(r, c, l)$ — robot r takes container c at location l

$$\begin{aligned} \text{Pre:} &\quad \texttt{loc}(r, l) \wedge \texttt{pos}(c, l) \wedge \neg\texttt{loaded}(r) \\ \text{Eff:} &\quad \texttt{pos}(c, r) \wedge \neg\texttt{pos}(c, l) \wedge \texttt{loaded}(r) \end{aligned}$$

Action 2: $\texttt{move}(r, l, m)$ — robot r moves from location l to location m

$$\begin{aligned} \text{Pre:} &\quad \texttt{loc}(r, l) \\ \text{Eff:} &\quad \texttt{loc}(r, m) \wedge \neg\texttt{loc}(r, l) \end{aligned}$$

Action 3: $\texttt{put}(r, c, l)$ — robot r puts down container c at location l

$$\begin{aligned} \text{Pre:} &\quad \texttt{loc}(r, l) \wedge \texttt{pos}(c, r) \\ \text{Eff:} &\quad \texttt{pos}(c, l) \wedge \neg\texttt{pos}(c, r) \wedge \neg\texttt{loaded}(r) \end{aligned}$$

6.4.2 State Variable Representation

In the state variable representation, we use functional notation with assignments:

Action 1: $\texttt{take}(r, c, l)$

$$\begin{aligned} \text{Pre:} &\quad \texttt{loc}(r) = l \wedge \texttt{pos}(c) = l \wedge \neg\texttt{loaded}(r) \\ \text{Eff:} &\quad \texttt{pos}(c) \leftarrow r \wedge \texttt{loaded}(r) \end{aligned}$$

Action 2: $\texttt{move}(r, l, m)$

$$\begin{aligned} \text{Pre:} &\quad \texttt{loc}(r) = l \\ \text{Eff:} &\quad \texttt{loc}(r) \leftarrow m \end{aligned}$$

Action 3: $\texttt{put}(r, c, l)$

$$\begin{array}{ll} \text{Pre:} & \texttt{loc}(r) = l \wedge \texttt{pos}(c) = r \\ \text{Eff:} & \texttt{pos}(c) \leftarrow l \wedge \neg\texttt{loaded}(r) \end{array}$$

6.4.3 Key Differences

- **Classical**: Requires explicit deletion of old values (e.g., $\neg\texttt{loc}(r, l)$ when moving)
- **State Variable**: Implicit deletion through reassignment (e.g., $\texttt{loc}(r) \leftarrow m$ automatically removes the old location)
- **Compactness**: State variables are more concise for multi-valued attributes

7 Planning as Satisfiability (SAT)

7.1 Recap: Core Concepts

Before introducing the SAT encoding, let us recap the fundamental concepts:

- **State**: A Boolean assignment to propositional (ground) literals
- **Action**: Characterized by:
  - **Preconditions**: A set (or conjunction) of literals
  - **Effects**: A set (or conjunction) of literals

7.2 Temporal Instantiation of Variables

To encode planning problems as SAT, we introduce a temporal dimension to our propositional variables.

Temporal Instantiation

Definition 7.1 (Temporal Instantiation)**.** Let X be the set of propositional variables describing the domain. The **temporal instantiation** of X at time t is:

$$X@t = \{x@t \mid x \in X\} \quad \text{for } t \geq 0$$

where $x@t$ is a propositional variable representing the truth value of x at time step t.

Intuition: Since a plan is a sequence of actions $(a_0, a_1, a_2, \ldots, a_{n-1})$ of length n, we need to track the truth value of each proposition at each time step from 0 to n.

7.3 The Bounded Planning Problem

Bounded Planning Problem

Definition 7.2 (Bounded Planning Problem)**.** Given a planning problem $P = \langle \Sigma, I, G \rangle$, the **bounded planning problem** asks:
Does there exist a plan solving P with length exactly n?

This is a decision problem parameterized by the plan length n.

7.4 SAT Encoding

The bounded planning problem can be encoded as a propositional satisfiability problem.

Planning as SAT

Theorem 7.1 (Planning as SAT)**.** *For a planning problem P and bound n, there exists a propositional formula Φ_n such that:*

$$\Phi_n \text{ is satisfiable} \quad \Leftrightarrow \quad \text{there exists a plan of length } n \text{ for } P$$

The formula Φ_n encodes:

1. **Initial state**: $I@0$ (the initial state holds at time 0)
2. **Goal state**: $G@n$ (the goal holds at time n)
3. **Action preconditions and effects**: For each time step $t \in \{0, 1, \ldots, n-1\}$, exactly one action is executed and its effects are correctly applied
4. **Frame axioms**: Variables not affected by actions remain unchanged

7.5 Solving Planning via Iterative Deepening

Since we don't know the optimal plan length in advance, we use **iterative deepening**:

Iterative Deepening SAT Planning

Algorithm 1 Planning via SAT with Iterative Deepening

```
n ← 1
while true do
    Construct formula Φ_n
    if Φ_n is satisfiable then
        Extract plan from satisfying assignment
        return plan
    end if
    n ← n + 1
end while
```

Process:

1. Try $n = 1$: Check if Φ_1 is satisfiable
2. If not, try $n = 2$: Check if Φ_2 is satisfiable
3. If not, try $n = 3$: Check if Φ_3 is satisfiable
4. Continue until a satisfiable formula is found

Advantages:

- Finds shortest plans (optimal in number of steps)
- Leverages highly optimized SAT solvers
- Scales well for many planning domains

Challenges:

- Formula size grows with n
- May need to try many values of n before finding a solution
- Encoding frame axioms can be expensive

Part III

SAT Solvers and Planning

8 The Boolean Satisfiability Problem

8.1 Definition and Scope

The **Boolean Satisfiability Problem (SAT)** is one of the most fundamental problems in computer science and forms the foundation for many automated reasoning techniques, including planning.

SAT Problem

Definition 8.1 (SAT Problem)**.** The SAT problem asks: Given a propositional formula, does there exist an interpretation (truth assignment) that makes the formula true? Formally, given a set of clauses S in propositional logic, determine whether S is satisfiable.

8.2 Propositional Logic Basics

To understand SAT, we must first establish the basic elements of propositional logic:

- **Propositional variables**: Let X be a finite set of propositional variables, where each variable can assume one of two values: `true` (1) or `false` (0)
- **Atom (Atomic formula)**: A single propositional variable $x \in X$
- **Literal**: Either an atom x or its negation $\neg x$
- **Clause**: A disjunction of literals: $L_1 \vee L_2 \vee \cdots \vee L_n$ where each L_i is a literal

8.3 Interpretations and Truth Assignments

Interpretation

Definition 8.2 (Interpretation)**.** An **interpretation** (or **truth assignment**) is a function:

$$v : X \to \{0, 1\}$$

that assigns a Boolean value to each propositional variable.

Number of interpretations: If $|X| = n$ (i.e., we have n propositional variables), then there are exactly 2^n possible interpretations.

8.3.1 Satisfiability of Clauses

Satisfaction

Definition 8.3 (Clause Satisfaction)**.** A clause $C = L_1 \vee L_2 \vee \cdots \vee L_n$ is **satisfied** by an interpretation v if there exists at least one literal L_i such that $v(L_i) = \text{true}$ for some $i \in \{1, 2, \ldots, n\}$.
We write: $v \models C$
Definition 8.4 (Formula Satisfaction)**.** A set of clauses $S = \{C_1, C_2, \ldots, C_m\}$ is **satisfiable** if there exists an interpretation v that satisfies all clauses in S:

$$v \models S \quad \Leftrightarrow \quad v \models C_i \text{ for all } i \in \{1, 2, \ldots, m\}$$

8.4 SAT Decision Procedure

The SAT problem can be viewed as a decision procedure with the following input-output specification:

$$S \text{ (set of clauses)} \xrightarrow{\text{SAT Solver}} \begin{cases} \textbf{YES (Satisfiable)} \\ \exists v : v \models S \\ \\ \textbf{NO (Unsatisfiable)} \\ \text{Proof: } S \vdash \square \end{cases}$$

Output cases:

1. **YES (Satisfiable)**: There exists an interpretation v such that $v \models S$. The SAT solver typically returns this satisfying assignment.
2. **NO (Unsatisfiable)**: No interpretation satisfies S. The solver may provide a proof of unsatisfiability, showing that S derives a contradiction (denoted $S \vdash \square$, where $\square$ represents the empty clause).

8.5 The Empty Clause

Empty Clause

Definition 8.5 (Empty Clause)**.** The **empty clause**, denoted $\square$ (or $\square$), is a clause with no literals. It is **false in all interpretations** and therefore represents a contradiction.

Significance: If a SAT solver derives the empty clause from the input formula S, this constitutes a proof that S is unsatisfiable, since deriving a contradiction means no interpretation can satisfy all clauses.

9 Planning Problem Formalization

We formalized the planning problem with:

$$P = \langle \Sigma, s_0, S_g \rangle$$

where $\Sigma = \langle S, A, \gamma \rangle$.

- s_0 is the initial state

- S_g is the set of goal states: $S_g = \{s \mid s \models G\}$
- G is the goal formula (a conjunction of atoms)

9.1 States as Interpretations

States correspond to interpretations in propositional logic:

- $\texttt{at}(r_1, l_1)$: ground positive literal
- $\neg\texttt{loaded}(r_1)$: ground negative literal

Abstraction: From ground first-order literals to propositional literals. With finitely many ground atoms (literals) in the planning problem, we obtain finitely many variables $|X| = n$.

10 Bounded Planning Problem

Definition 10.1 (Bounded Planning Problem)**.** Does there exist a plan π that solves planning problem P and has length k (i.e., $|\pi| = k$, where π is a sequence of k actions)?

10.1 Iterative Deepening on Plan Length

We use iterative deepening to find the shortest plan by increasing the bound k:

$$P \xrightarrow{\text{translation}} \Phi \xrightarrow{\text{SAT}} \begin{cases} \textbf{YES} : \text{interpretation } v \models \Phi \\ \text{encodes plan } \pi \text{ of length } k \\ \textbf{NO} : \text{no plan of length } k \text{ solves } P \end{cases}$$

The process iterates: $k := k + 1$ until a solution is found.

Key insight: The SAT encoding Φ includes:

- Initial state constraints at time 0
- Goal constraints at time k
- Action preconditions and effects for each time step
- Frame axioms ensuring persistence of unchanged literals

10.2 Temporal Encoding of Literals

A key aspect of the SAT encoding for planning is the **temporal instantiation** of literals. Since a plan of length k requires tracking state changes over time, each literal is indexed by time steps.

- **Plan length** k means we have $0 \leq i \leq k - 1$ time steps (transitions)
- Each literal f becomes $f@t$ where t ranges from 0 to k
- State i is the state after executing i actions (before the i-th action)

10.2.1 Example: Temporal Instantiation

Consider the literal $\texttt{at}(r_1, l_1)$ ("robot r_1 is at location l_1"):

$$\texttt{at}(r_1, l_1) \quad \xrightarrow{\text{temporal encoding}} \quad \texttt{at}(r_1, l_1, i)$$

Similarly, the negative literal $\neg\texttt{loaded}(r_1)$ becomes $\neg\texttt{loaded}(r_1, i)$, meaning "robot r_1 is not loaded at stage i".

State Transition

Definition 10.2 (State Transition)**.** Step i (where $0 \leq i \leq k-1$) takes us from state i to state $i+1$ by executing one action.

10.3 Encoding the Initial State

The initial state s_0 is encoded as a conjunction of constraints at time $t = 0$:

$$\Phi_{s_0} = \bigwedge_{f \in s_0} f@0 \quad \wedge \quad \bigwedge_{f \notin s_0} \neg f@0$$

Where f represents ground literals.

10.3.1 From Literals to Propositional Variables

Each temporally indexed literal corresponds to a propositional variable:

- $\texttt{at}(r_1, l_1, 0)$ corresponds to variable $x_{\texttt{at}(r_1,l_1)}@0$
- $\neg\texttt{loaded}(r_1, 0)$ corresponds to $\neg x_{\texttt{loaded}(r_1)}@0$

10.3.2 Example: Complete Initial State Encoding

Consider an initial state containing $\texttt{at}(r_1, l_1)$ but not $\texttt{loaded}(r_1)$:

Literals in s_0 :	$\texttt{at}(r_1, l_1)$	$\wedge\neg\texttt{loaded}(r_1) \wedge \cdots$
Temporal encoding:	$\texttt{at}(r_1, l_1, 0)$	$\wedge\neg\texttt{loaded}(r_1, 0) \wedge \cdots$
SAT variables:	$x_{\texttt{at}(r_1,l_1)}@0$	$\wedge\neg x_{\texttt{loaded}(r_1)}@0 \wedge \cdots$

For literals not in the initial state (e.g., $\texttt{at}(r_1, l_2)$, $\texttt{at}(r_1, l_3)$), these are assigned false:

$$\neg\texttt{at}(r_1, l_2, 0), \quad \neg\texttt{at}(r_1, l_3, 0)$$

Alternative notation: The initial state encoding can also be written using set notation:

$$\Phi_{s_0} = \bigwedge_{x \in S_0^+} x_0 \wedge \bigwedge_{x \in S_0^-} \neg x_0$$

where S_0^+ is the set of positive literals in s_0 and S_0^- is the set of negative literals in s_0.

10.4 Encoding the Goal States

The goal conditions must be satisfied at the final time step k.

Goal Encoding

Definition 10.3 (Goal Encoding)**.** The goal states are encoded at time k as:

$$\Phi_G = \bigwedge_{x \in G^+} x_k \wedge \bigwedge_{x \in G^-} \neg x_k$$

where G^+ is the set of positive literals in the goal G and G^- is the set of negative literals in G.

Interpretation: This formula ensures that all positive goal conditions are true and all negative goal conditions are false in the final state after executing k actions.

10.5 Encoding Actions

The planning problem is defined as $\Sigma = \langle S, A, \gamma \rangle$, where A is the set of all ground instances of the action schemas.

10.5.1 Action Preconditions

For each action $a \in A$ at each time step i (where $0 \leq i \leq k-1$), we encode the constraint that if the action is executed, its preconditions must hold:

Action Precondition Encoding

Definition 10.4 (Action Precondition Encoding)**.**

$$\forall a \in A, \forall i \in \{0, \ldots, k-1\} : \quad a_i \rightarrow \bigwedge_{p \in \text{prec}(a)} p_i$$

where $\text{prec}(a)$ denotes the set of preconditions of action a.

Interpretation: This formula states that if action a is executed at time i, then all of its preconditions must be satisfied at time i. The implication can be rewritten in CNF as:

$$\neg a_i \vee \bigwedge_{p \in \text{prec}(a)} p_i$$

10.5.2 Mutual Exclusion of Actions

At each time step, at most one action can be executed. This constraint is encoded as follows:

Action Mutual Exclusion

Definition 10.5 (Action Mutual Exclusion)**.** For all pairs of distinct actions $(a, b) \in A \times A$ where $a \neq b$, and for all time steps i where $0 \leq i \leq k-1$:

$$\neg a_i \vee \neg b_i$$

This formula ensures that actions a and b cannot both be executed at the same time step i.

Equivalent formulations:

- $a_i \rightarrow \neg b_i$: If action a is executed at time i, then action b is not executed
- $b_i \rightarrow \neg a_i$: If action b is executed at time i, then action a is not executed

10.6 Complete SAT Encoding

The complete SAT encoding Φ for a bounded planning problem with plan length k is the conjunction of the following sets of propositional formulas:

$$\Phi = \Phi_{s_0} \wedge \Phi_G \wedge \Phi_{\text{actions}} \wedge \Phi_{\text{mutex}} \wedge \Phi_{\text{frame}}$$

where:

- Φ_{s_0}: Initial state constraints
- Φ_G: Goal state constraints
- Φ_{actions}: Action precondition and effect constraints
- Φ_{mutex}: Mutual exclusion constraints between actions
- Φ_{frame}: Frame axioms (persistence of unchanged literals)

10.7 Plan Extraction from SAT Model

Once the SAT solver returns a satisfying assignment, we can extract the corresponding plan.

Plan Extraction

Definition 10.6 (Plan Extraction)**.** Let v be a satisfying interpretation (model) returned by the SAT solver, represented as a set of literals. The plan π is extracted as follows:

$$\pi = \langle a_0, a_1, \ldots, a_{k-1} \rangle$$

where a_i is the unique action such that $v \models a_i$ (i.e., a_i is assigned `true` in the model v) for each time step $i \in \{0, 1, \ldots, k-1\}$.

Extraction procedure:

1. Filter the model v to obtain only the action literals: $\{a_i \mid a \in A, 0 \leq i \leq k-1, v \models a_i\}$
2. For each time step i, identify the action a such that a_i is true in v
3. Construct the sequence $\pi = \langle a_0, a_1, \ldots, a_{k-1} \rangle$

Remark. Due to the mutual exclusion constraints, at most one action is true at each time step. If no action is true at some time step i, this corresponds to a "no-op" (no operation) action.

Part IV

SAT Encoding Examples

11 Detailed SAT Encoding Example

11.1 Simple Robot Navigation Domain

To illustrate the complete SAT encoding process, we present a simple example with a single robot and two locations.

Example 11.1 (Robot Navigation)**.** Consider a minimal planning domain with the following components:

Objects:

- One robot: r_1
- Two locations: l_1, l_2

Action schema: $\texttt{move}(r, l, l')$

Parameters:	r (robot), l (source location), l' (destination location)
Precondition:	$\texttt{at}(r, l)$
Effect:	$\texttt{at}(r, l') \land \neg\texttt{at}(r, l)$

11.1.1 Ground Actions

With one robot and two locations, we obtain two ground actions:

1. $a_1 = \texttt{move}(r_1, l_1, l_2)$: Move robot r_1 from location l_1 to location l_2
2. $a_2 = \texttt{move}(r_1, l_2, l_1)$: Move robot r_1 from location l_2 to location l_1

11.1.2 Problem Instance

Initial state (at time step $t = 0$):

$$s_0 = \{\texttt{at}(r_1, l_1)\}$$

The robot starts at location l_1. This is encoded temporally as:

$$\texttt{at}(r_1, l_1)@0$$

Goal state (at time step $t = 1$):

$$G = \{\texttt{at}(r_1, l_2)\}$$

The goal is for the robot to be at location l_2 after one action. This is encoded as:

$$\texttt{at}(r_1, l_2)@1$$

11.2 Complete SAT Encoding

For this problem with plan length $k = 1$, we construct the following propositional formulas:

11.2.1 Initial State Constraints

$$\Phi_{s_0} = \texttt{at}(r_1, l_1)@0 \land \neg\texttt{at}(r_1, l_2)@0$$

11.2.2 Goal State Constraints

$$\Phi_G = \texttt{at}(r_1, l_2)@1$$

11.2.3 Action Preconditions and Effects

For action $a_1 = \texttt{move}(r_1, l_1, l_2)$ at time step 0:

$$\texttt{move}(r_1, l_1, l_2)@0 \rightarrow \big(\texttt{at}(r_1, l_1)@0 \land \texttt{at}(r_1, l_2)@1 \land \neg\texttt{at}(r_1, l_1)@1\big)$$

For action $a_2 = \texttt{move}(r_1, l_2, l_1)$ at time step 0:

$$\texttt{move}(r_1, l_2, l_1)@0 \rightarrow \big(\texttt{at}(r_1, l_2)@0 \land \texttt{at}(r_1, l_1)@1 \land \neg\texttt{at}(r_1, l_2)@1\big)$$

11.2.4 Explanatory Frame Axioms

The **explanatory frame axioms** (also called **explanation axioms**) state that if a literal changes its truth value between time steps, then some action causing that change must have been executed.

If $\texttt{at}(r_1, l_1)$ becomes false:

$$\texttt{at}(r_1, l_1)@0 \land \neg\texttt{at}(r_1, l_1)@1 \rightarrow \texttt{move}(r_1, l_1, l_2)@0$$

If $\texttt{at}(r_1, l_2)$ becomes false:

$$\texttt{at}(r_1, l_2)@0 \land \neg\texttt{at}(r_1, l_2)@1 \rightarrow \texttt{move}(r_1, l_2, l_1)@0$$

If $\texttt{at}(r_1, l_1)$ becomes true:

$$\neg\texttt{at}(r_1, l_1)@0 \land \texttt{at}(r_1, l_1)@1 \rightarrow \texttt{move}(r_1, l_2, l_1)@0$$

If $\texttt{at}(r_1, l_2)$ becomes true:

$$\neg\texttt{at}(r_1, l_2)@0 \land \texttt{at}(r_1, l_2)@1 \rightarrow \texttt{move}(r_1, l_1, l_2)@0$$

11.3 Solution Extraction

Given the constraints, the SAT solver will find a satisfying assignment that includes:

$$\texttt{move}(r_1, l_1, l_2)@0 = \text{true}$$

The extracted plan is:

$$\pi = \langle \texttt{move}(r_1, l_1, l_2) \rangle$$

This plan has length 1 and achieves the goal of moving the robot from l_1 to l_2.

12 Conversion to Conjunctive Normal Form

12.1 CNF Definition and Structure

To use a SAT solver, we must convert all formulas to **Conjunctive Normal Form (CNF)**.

Conjunctive Normal Form (CNF)

Definition 12.1 (Clause)**.** A **clause** is a disjunction of literals: $L_1 \vee L_2 \vee \cdots \vee L_k$ where each L_i is a literal (an atom or its negation).
Definition 12.2 (Conjunctive Normal Form (CNF))**.** A formula is in **Conjunctive Normal Form** if it is a conjunction of clauses:

$$\text{CNF} = (L_1^1 \vee L_2^1 \vee \cdots \vee L_k^1) \wedge (L_1^2 \vee L_2^2 \vee \cdots \vee L_k^2) \wedge \cdots \wedge (L_1^m \vee L_2^m \vee \cdots \vee L_k^m)$$

A set of clauses is logically equivalent to a conjunction of those clauses, which is precisely the CNF representation required by SAT solvers.

12.2 Standard CNF Transformation

The standard transformation to CNF involves applying logical equivalences to eliminate implications and biconditionals, push negations inward, and distribute disjunctions over conjunctions.

12.2.1 Logical Equivalences

The following logical equivalences are used in the transformation:

Biconditional elimination:

$$F \leftrightarrow G \equiv (F \rightarrow G) \wedge (G \rightarrow F)$$

Implication elimination:

$$F \rightarrow G \equiv \neg F \vee G$$

De Morgan's laws:

$$\begin{aligned} \neg(F \vee G) &\equiv \neg F \wedge \neg G \\ \neg(F \wedge G) &\equiv \neg F \vee \neg G \end{aligned}$$

12.2.2 Negation Normal Form (NNF)

After applying the above equivalences, we obtain a formula in **Negation Normal Form (NNF)**, where negations appear only directly in front of atoms (not in front of complex formulas).

12.2.3 Distribution to Obtain CNF

To convert from NNF to CNF, we apply the **distributive law**:

$$F \vee (G \wedge H) \equiv (F \vee G) \wedge (F \vee H)$$

This distributes disjunctions over conjunctions, eventually producing a conjunction of disjunctions (CNF).

12.3 The Exponential Blowup Problem

Critical issue: The standard CNF transformation can cause a **combinatorial explosion** in formula size.

Exponential Blowup Problem

Observation (Exponential Blowup). In the worst case, the standard transformation into CNF produces a formula whose size is **exponential** in the size of the original formula: $O(2^n)$ where n is the size of the input formula.

This exponential blowup makes the standard transformation **impractical** for large planning problems, as the resulting CNF formula becomes too large to handle efficiently.

12.4 Equisatisfiable CNF (ECNF)

To avoid the exponential blowup, we use a different approach that produces an **Equisatisfiable CNF (ECNF)**.

Equisatisfiable CNF (ECNF)

Definition 12.3 (Equisatisfiable CNF)**.** An **Equisatisfiable CNF (ECNF)** is a CNF formula that is not logically equivalent to the original formula, but is **equisatisfiable**—it has the same satisfiability status:

- The original formula is satisfiable if and only if the ECNF is satisfiable
- The ECNF may contain additional auxiliary variables not present in the original formula

12.4.1 Tseitin Encoding

The standard technique for producing ECNF is the **Tseitin encoding** (also called **Tseitin transformation**), named after Grigori Tseitin.

Key idea: Introduce auxiliary variables to represent subformulas, avoiding the need for distribution.

Tseitin Encoding Complexity

Theorem 12.1 (Tseitin Encoding Complexity)**.** *The Tseitin encoding produces an ECNF formula whose size is **linear** in the size of the original formula: $O(n)$ where n is the size of the input formula.*

Advantages of ECNF:

- **Linear size**: Avoids exponential blowup
- **Preserves satisfiability**: The ECNF is satisfiable if and only if the original formula is satisfiable
- **Efficient encoding**: Can be computed in polynomial time
- **Practical for planning**: Makes SAT-based planning feasible for large problems

Trade-off:

- The ECNF introduces additional variables (one for each subformula)
- The ECNF is not logically equivalent to the original formula (only equisatisfiable)

- However, the satisfying assignments to the original variables correspond to solutions of the original formula

12.5 Tseitin Encoding: Formal Construction

The Tseitin encoding is based on two key functions that work together to produce the ECNF.

12.5.1 The Representative Function

Representative Function

Definition 12.4 (Representative Function)**.** Let PL denote the language of propositional formulas over a set of propositional variables X, and let V be a set of fresh (new) propositional variables. The **representative function** is defined as:

$$\text{Rep} : \text{PL} \to V \cup \{\bot, \top\}$$

The function Rep assigns to each formula a representative variable (or constant). It is defined inductively:

Base case (atomic formulas):

$$\begin{array}{lcll} \text{Rep}(x) & = & x & \text{for all } x \in X \\ \text{Rep}(\bot) & = & \bot & \\ \text{Rep}(\top) & = & \top & \end{array}$$

Inductive case (complex formulas):

$$\text{Rep}(F) = v_F$$

where $F \in \text{PL}$ is a non-atomic formula (i.e., F is neither a propositional variable nor $\bot$ nor $\top$), and $v_F \in V$ is a fresh propositional variable uniquely associated with F.

Intuition: Each complex subformula is represented by a new auxiliary variable. This avoids the need to duplicate subformulas during the transformation.

12.5.2 The Encoding Function

Encoding Function

Definition 12.5 (Encoding Function)**.** The **encoding function** is defined as:

$$\text{En} : \text{PL} \to \text{PL}$$

$$\text{En}(F) = F \leftrightarrow \text{Rep}(F)$$

The encoding function creates a biconditional formula stating that formula F is logically equivalent to its representative.

Interpretation: $\text{En}(F)$ encodes the constraint that the auxiliary variable $\text{Rep}(F)$ must have the same truth value as the formula F itself.

12.5.3 Complete ECNF Construction

ECNF via Tseitin Encoding

Definition 12.6 (ECNF via Tseitin Encoding)**.** Given an input formula F, let S_F denote the set of all subformulas of F (including F itself). The **Equisatisfiable CNF** of F is:

$$\text{ECNF}(F) = \text{Rep}(F) \wedge \bigwedge_{G \in S_F} \text{En}(G)$$

Components:

- $\text{Rep}(F)$: The representative variable for the entire formula (must be true)
- $\bigwedge_{G \in S_F} \text{En}(G)$: Conjunction of encoding constraints for all subformulas

Why "Equisatisfiable"?

The transformation from F to $\text{ECNF}(F)$ does **not** preserve logical equivalence, but it **does** preserve satisfiability:

Equisatisfiability Theorem

Theorem 12.2 (Equisatisfiability)**.** *Formula F is satisfiable if and only if ECNF(F) is satisfiable.*

Proof sketch. **Forward direction** (F satisfiable $\Rightarrow$ $\text{ECNF}(F)$ satisfiable): If F is satisfiable, we can extend the satisfying assignment to include the auxiliary variables by setting each $v_G = \text{value}(G)$ under the original assignment. This satisfies all encoding constraints.

Backward direction ($\text{ECNF}(F)$ satisfiable $\Rightarrow$ F satisfiable): If $\text{ECNF}(F)$ is satisfiable, the encoding constraints ensure that $\text{Rep}(F)$ has the same truth value as F. Since $\text{Rep}(F)$ must be true (it appears as a conjunct), F must also be true under the restriction of the assignment to the original variables. □

Key insight: The ECNF introduces auxiliary variables but maintains the same satisfiability status as the original formula, while keeping the formula size linear.

12.6 Example: Equisatisfiability vs. Logical Equivalence

To illustrate the difference between equisatisfiability and logical equivalence, consider a concrete example.

Example 12.1 (Tseitin Encoding of Disjunction)**.** Consider the formula:

$$F = A \vee (B \wedge C)$$

where A, B, and C are propositional variables.

Using the standard CNF transformation (distribution), we would obtain:

$$F_{\text{CNF}} = (A \vee B) \wedge (A \vee C)$$

This is **logically equivalent** to F, but for larger formulas, this approach leads to exponential blowup.

12.6.1 Tseitin Encoding Approach

Instead, we apply the Tseitin encoding by introducing an auxiliary variable v to represent the subformula $B \wedge C$:

Step 1: Define the encoding constraint:

$$v \leftrightarrow (B \wedge C)$$

Step 2: Replace the subformula in the original formula:

$$A \vee v$$

Step 3: Convert the biconditional to CNF. The biconditional $v \leftrightarrow (B \wedge C)$ expands to:

$$(v \rightarrow (B \wedge C)) \wedge ((B \wedge C) \rightarrow v)$$

Converting each implication:

$$v \rightarrow (B \wedge C) \equiv \neg v \vee (B \wedge C) \equiv (\neg v \vee B) \wedge (\neg v \vee C)$$
$$(B \wedge C) \rightarrow v \equiv \neg(B \wedge C) \vee v \equiv \neg B \vee \neg C \vee v$$

Final ECNF: The complete set of clauses is:

$$S = \{(A \vee v),\ (\neg v \vee B),\ (\neg v \vee C),\ (\neg B \vee \neg C \vee v)\}$$

Numbering the clauses:

$$\begin{array}{ll} (1) & A \vee v \\ (2) & \neg v \vee B \\ (3) & \neg v \vee C \\ (4) & \neg B \vee \neg C \vee v \end{array}$$

12.6.2 Demonstrating Equisatisfiability

Consider the interpretation:

$$I = \{A \mapsto \text{true},\ B \mapsto \text{false},\ C \mapsto \text{false}\}$$

Evaluating F under I:

$$F = A \vee (B \wedge C) = \text{true} \vee (\text{false} \wedge \text{false}) = \text{true} \vee \text{false} = \text{true}$$

Therefore, $I \models F$ (interpretation I satisfies F).

Evaluating S under I:

We need to extend I to assign a value to the auxiliary variable v. Let's first try without extending:

- Clause (1): $A \vee v = \text{true} \vee v = \text{true}$ (satisfied regardless of v)
- Clause (2): $\neg v \vee B = \neg v \vee \text{false} = \neg v$ (satisfied only if $v = \text{false}$)
- Clause (3): $\neg v \vee C = \neg v \vee \text{false} = \neg v$ (satisfied only if $v = \text{false}$)

- Clause (4): $\neg B \vee \neg C \vee v = \text{true} \vee \text{true} \vee v = \text{true}$ (satisfied regardless of v)

For I to satisfy S, we must set $v = \text{false}$ (to satisfy clauses (2) and (3)). With this extension:

$$I' = \{A \mapsto \text{true}, B \mapsto \text{false}, C \mapsto \text{false}, v \mapsto \text{false}\}$$

Now $I' \models S$ (all clauses are satisfied).

Equisatisfiability vs Logical Equivalence

Observation (Not Logically Equivalent). The original interpretation I (without the auxiliary variable) satisfies F but does **not** satisfy all clauses in S unless extended with the appropriate value for v. This demonstrates that F and S are **not logically equivalent**. However, F is satisfiable if and only if S is satisfiable (by appropriately choosing the value of v). Therefore, F and S are **equisatisfiable**.

Conclusion: The Tseitin encoding produces a formula that is equisatisfiable with the original but not logically equivalent. This is acceptable for SAT-based planning because we only care about whether a solution exists, not about preserving all logical properties.

12.7 Detailed Definition of the Encoding Function

We now provide the complete formal definition of the encoding function En for all cases.

Definition 12.7 (Encoding Function - Complete Definition)**.** The encoding function $\text{En} : \text{PL} \to \text{PL}$ is defined inductively as follows:

Base cases (atomic formulas):

$$\begin{aligned} \text{En}(\bot) &= \top \\ \text{En}(\top) &= \top \\ \text{En}(x) &= \top \quad \text{for all } x \in X \end{aligned}$$

Inductive case for negation:

$$\begin{aligned} \text{En}(\neg F) &= \text{let } v = \text{Rep}(\neg F) \text{ in } v \leftrightarrow \neg \text{Rep}(F) \\ &= (v \to \neg \text{Rep}(F)) \wedge (\neg \text{Rep}(F) \to v) \\ &= (\neg v \vee \neg \text{Rep}(F)) \wedge (\text{Rep}(F) \vee v) \end{aligned}$$

Inductive case for binary connectives:

For all binary connectives $\circ \in \{\wedge, \vee, \to, \leftrightarrow\}$:

$$\text{En}(F_1 \circ F_2) = \text{let } v = \text{Rep}(F_1 \circ F_2) \text{ in } v \leftrightarrow (\text{Rep}(F_1) \circ \text{Rep}(F_2))$$

12.7.1 Example: Conjunction Encoding

Example 12.2 (Encoding Conjunction)**.** Consider the encoding of $F_1 \wedge F_2$:

$$\begin{aligned} \text{En}(F_1 \wedge F_2) &= \text{let } v = \text{Rep}(F_1 \wedge F_2) \text{ in } v \leftrightarrow (\text{Rep}(F_1) \wedge \text{Rep}(F_2)) \\ &= (v \to (\text{Rep}(F_1) \wedge \text{Rep}(F_2))) \wedge ((\text{Rep}(F_1) \wedge \text{Rep}(F_2)) \to v) \\ &= (\neg v \vee (\text{Rep}(F_1) \wedge \text{Rep}(F_2))) \wedge ((\text{Rep}(F_1) \wedge \text{Rep}(F_2)) \vee v) \end{aligned}$$

Distributing to obtain CNF:

$$(\neg v \vee \text{Rep}(F_1)) \wedge (\neg v \vee \text{Rep}(F_2)) \wedge (\text{Rep}(F_1) \vee v) \wedge (\text{Rep}(F_2) \vee v)$$

This produces **4 clauses** in CNF.

12.8 Complexity Analysis of Tseitin Encoding

We now analyze the size of the resulting ECNF formula to prove linear complexity.

Linear Size of ECNF

Theorem 12.3 (Linear Size of ECNF)**.** *Let F be a propositional formula with size $|F| = n$. The Tseitin encoding produces an ECNF formula:*

$$F' = \mathit{Rep}(F) \wedge \bigwedge_{G \in S_F} \mathit{En}(G)$$

where S_F is the set of all subformulas of F (including F itself). The size of F' is $|F'| = O(n)$.

12.8.1 Worst Case Analysis: Biconditional

The worst case for the encoding function occurs with the biconditional connective. Let us analyze:

Example 12.3 (Biconditional Encoding - Complete Derivation)**.** We show the complete step-by-step conversion of the biconditional encoding to CNF:

Step 1: Start with the encoding

$$\text{En}(F_1 \leftrightarrow F_2) = \text{let } v = \text{Rep}(F_1 \leftrightarrow F_2) \text{ in } v \leftrightarrow (\text{Rep}(F_1) \leftrightarrow \text{Rep}(F_2))$$

Step 2: Expand the inner biconditional

$$= v \leftrightarrow [(\text{Rep}(F_1) \to \text{Rep}(F_2)) \wedge (\text{Rep}(F_2) \to \text{Rep}(F_1))]$$

Step 3: Expand the outer biconditional

$$\begin{aligned} &= (v \to [(\text{Rep}(F_1) \to \text{Rep}(F_2)) \wedge (\text{Rep}(F_2) \to \text{Rep}(F_1))]) \\ &\quad \wedge ([(\text{Rep}(F_1) \to \text{Rep}(F_2)) \wedge (\text{Rep}(F_2) \to \text{Rep}(F_1))] \to v) \end{aligned}$$

Step 4: Convert implications to disjunctions

$$\begin{aligned} &= (\neg v \vee [(\text{Rep}(F_1) \to \text{Rep}(F_2)) \wedge (\text{Rep}(F_2) \to \text{Rep}(F_1))]) \\ &\quad \wedge (\neg[(\text{Rep}(F_1) \to \text{Rep}(F_2)) \wedge (\text{Rep}(F_2) \to \text{Rep}(F_1))] \vee v) \end{aligned}$$

Step 5: Expand inner implications

$$\begin{aligned} &= (\neg v \vee [(\neg\text{Rep}(F_1) \vee \text{Rep}(F_2)) \wedge (\neg\text{Rep}(F_2) \vee \text{Rep}(F_1))]) \\ &\quad \wedge (\neg[(\neg\text{Rep}(F_1) \vee \text{Rep}(F_2)) \wedge (\neg\text{Rep}(F_2) \vee \text{Rep}(F_1))] \vee v) \end{aligned}$$

Step 6: Distribute $\neg v$ over the conjunction (first part)

$$= (\neg v \vee \neg\text{Rep}(F_1) \vee \text{Rep}(F_2)) \wedge (\neg v \vee \neg\text{Rep}(F_2) \vee \text{Rep}(F_1))$$

Step 7: Apply De Morgan's law to the second part

$$\wedge \left(\neg(\neg\text{Rep}(F_1) \vee \text{Rep}(F_2)) \vee \neg(\neg\text{Rep}(F_2) \vee \text{Rep}(F_1)) \vee v\right)$$

Step 8: Apply De Morgan's law again

$$\wedge \left((\text{Rep}(F_1) \wedge \neg\text{Rep}(F_2)) \vee (\text{Rep}(F_2) \wedge \neg\text{Rep}(F_1)) \vee v\right)$$

Step 9: Distribute disjunction over conjunction

$$\begin{aligned}&\wedge (\text{Rep}(F_1) \vee \text{Rep}(F_2) \vee v) \wedge (\text{Rep}(F_1) \vee \neg\text{Rep}(F_1) \vee v)\\ &\wedge (\neg\text{Rep}(F_2) \vee \text{Rep}(F_2) \vee v) \wedge (\neg\text{Rep}(F_2) \vee \neg\text{Rep}(F_1) \vee v)\end{aligned}$$

Step 10: Simplify (remove tautologies)

The clauses $(\text{Rep}(F_1) \vee \neg\text{Rep}(F_1) \vee v)$ and $(\neg\text{Rep}(F_2) \vee \text{Rep}(F_2) \vee v)$ are tautologies and can be removed.

Final CNF: The encoding produces the following clauses:

$$\begin{aligned}&(\neg v \vee \neg\text{Rep}(F_1) \vee \text{Rep}(F_2))\\ &(\neg v \vee \neg\text{Rep}(F_2) \vee \text{Rep}(F_1))\\ &(\text{Rep}(F_1) \vee \text{Rep}(F_2) \vee v)\\ &(\neg\text{Rep}(F_2) \vee \neg\text{Rep}(F_1) \vee v)\end{aligned}$$

This produces **4 clauses** after simplification (or up to 8 clauses before removing tautologies).

Proof of Linear Complexity. **Key observations**:

1. The number of subformulas $|S_F|$ is at most n (the size of F)
2. Each subformula $G \in S_F$ contributes at most a constant number of clauses (at most 8 for biconditional)
3. Therefore, the total number of clauses is at most $8n$
4. Each clause has at most 3 literals
5. Total size: $|F'| \leq 8n \times 3 = 24n = O(n)$

More precisely, we can bound the size as $|F'| = O(30n + 2)$, which is still linear in n. □

Conclusion: The Tseitin encoding achieves linear size $O(n)$ instead of the exponential size $O(2^n)$ of the standard CNF transformation, making it practical for large planning problems.

Part V

SAT Decision Procedures

13 Decision Procedures for Satisfiability

We now present a **decision procedure** for determining the satisfiability of a set of clauses. This procedure is fundamental to understanding how SAT solvers work and forms the algorithmic foundation for planning via SAT encoding.

SAT Decision Problem

Definition 13.1 (SAT Decision Problem)**.** Given a set of clauses S over a set of Boolean variables X, determine whether there exists a model (satisfying assignment) $v : X \to \{\text{true}, \text{false}\}$ such that $v \models S$.

13.1 Relation to Constraint Solving

The SAT problem is closely related to **Constraint Solving Problems (CSP)** with Boolean variables. It represents one of the most fundamental problems in algorithmic theory and was the first problem proven to be **NP-complete**.

$$S \text{ (set of clauses)} \xrightarrow{\text{SAT Decision Procedure}} \begin{cases} \textbf{YES (Satisfiable)} \\ \exists v : v \models S \\ \\ \textbf{NO (Unsatisfiable)} \\ \text{with proof: } S \vdash \square \end{cases}$$

14 State Representation and Search Modes

14.1 Trail-Based State Representation

The state of the SAT decision procedure is represented using a **trail** of literals, which maintains the current partial assignment of truth values to variables.

Trail Representation

Definition 14.1 (Trail)**.** A **trail** Γ is a sequence of literals representing the current partial assignment in the search process. The empty trail is denoted ε.

Definition 14.2 (Search State)**.** A search state is represented as a pair $\langle S, \Gamma \rangle$ where:

- S is the current set of clauses
- Γ is the current trail (partial assignment)

14.2 Literal Status in Trails

Given a trail Γ and a literal L, the status of L is determined as follows:

- L is **true** if $L \in \Gamma$
- L is **false** if $\neg L \in \Gamma$

- L is **undefined** if neither L nor $\neg L$ appears in Γ

15 Transition Rules

The SAT decision procedure operates as a **transition system** with specific rules that transform the search state. There are two main types of transitions:

15.1 Decision Rule

The **decision rule** extends the trail by choosing an undefined literal and adding it to the current assignment.

Decision Rule

Definition 15.1 (Decision Transition)**.**

$$\langle S \parallel \Gamma \rangle \to \langle S \parallel \Gamma, L \rangle$$

where the decision is applicable if:

- $L \notin \Gamma$ (literal not already assigned)
- $\neg L \notin \Gamma$ (negation not already assigned)

The decision rule represents a **choice point** in the search process where the algorithm guesses a truth value for an undefined variable.

15.2 Propagation Rule

The **propagation rule** (also called **unit propagation**) adds literals that are logically implied by the current assignment.

Propagation Rule

Definition 15.2 (Propagation Transition)**.**

$$\langle S \parallel \Gamma \rangle \to \langle S \parallel \Gamma, L \rangle$$

where the propagation is applicable if there exists a clause $C = L_1 \vee L_2 \vee \cdots \vee L_{k-1} \vee L_k \in S$ such that:

- $D = L_1 \vee L_2 \vee \cdots \vee L_{k-1}$
- $\Gamma \models \neg D$ (all literals in D are false under current assignment)
- $L = L_k$ (the remaining literal becomes implied)
- $L \notin \Gamma$ and $\neg L \notin \Gamma$

15.2.1 Justification for Implied Literals

When a literal L_k is propagated with justification clause C, it means:

- The clause $C = L_1 \vee L_2 \vee \cdots \vee L_{k-1} \vee L_k$ must be satisfied
- All other literals $L_1, L_2, \ldots, L_{k-1}$ are currently false under Γ
- Therefore, L_k must be true to satisfy the clause

This is called the **justification** for the propagation: the clause C forces L_k to be true.

15.2.2 Logical Foundation of Propagation

The propagation rule is based on the following logical principle:

If we have a clause $D \vee L$ where $D = L_1 \vee L_2 \vee \cdots \vee L_{k-1}$, and we know that $\neg D$ (i.e., $\neg L_1 \wedge \neg L_2 \wedge \cdots \wedge \neg L_{k-1}$), then:

$$\neg D \models L$$

Because if all the alternative literals are false, the remaining literal must be true to satisfy the disjunction.

16 Solution States

The search process terminates when it reaches specific terminal states:

Solution States

- **Success**: The state $\langle S, \Gamma \rangle$ where all clauses in S are satisfied by the assignment represented by Γ
- **Failure**: The state where a conflict is detected (e.g., the empty clause $\square$ is derived)

The decision procedure systematically explores the space of possible assignments using these transition rules until it either finds a satisfying assignment or proves that no such assignment exists.

17 Conflict Analysis and Learning

When the propagation rule leads to a conflict (derivation of the empty clause $\square$), the SAT solver enters **conflict analysis mode** to learn from the conflict and guide future search.

17.1 Conflict-Solving Mode

Conflict State Representation

Definition 17.1 (Conflict State)**.** A state in conflict-solving mode is represented as a triple $\langle S \parallel \Gamma \parallel C \rangle$ where:

- S is the current set of clauses
- Γ is the current trail (partial assignment)
- C is the **conflict clause** that caused the conflict

The conflict clause C represents a learned constraint that explains why the current partial assignment leads to a contradiction. This clause can be added to the formula to prevent the same conflict in future search branches.

17.2 Search Space Complexity

The SAT decision procedure explores a search space that grows exponentially with the number of variables.

Search Space Size

Definition 17.2 (Search Space Complexity)**.** Given a SAT problem with n Boolean variables, the total number of possible assignments is:

$$2^n$$

where each variable can be assigned either true or false.

17.2.1 Search Tree Structure

The search process forms a binary tree where each level corresponds to a variable assignment. For two variables p and Q, the search space has the following structure:

- **Level 0**: Start state
- **Level 1**: Variable p can be assigned 1 or 0
- **Level 2**: For each assignment of p, variable Q can be assigned 1 or 0

This creates four possible complete assignments:

- $p = 1, Q = 1$
- $p = 1, Q = 0$
- $p = 0, Q = 0$
- $p = 0, Q = 1$

For n variables, this creates a complete binary tree of depth n with 2^n leaves, representing all possible assignments.

18 Backtracking Mechanism

When a conflict is detected or a dead-end is reached, the solver uses **backtracking** to explore alternative assignments.

18.1 Backtracking Rule

Backtracking Transition

Definition 18.1 (Backtracking)**.**

$$\langle S \parallel M\, L^d\, N \parallel C \rangle \to \langle S \parallel M\, \neg L^d \rangle$$

where:
- M is the sequence of literals before the decision
- L^d is the most recent **decided literal** (marked with d)
- N is the sequence of literals after the decision
- The transition is only applicable if N contains no decided literals

Backtracking implements a **depth-first search strategy** that:

- Explores one branch of the search tree completely before trying alternatives
- When a conflict occurs, backtracks to the most recent decision point

- Flips the decision (from L^d to $\neg L^d$) and continues search from there

18.2 Decision vs. Propagated Literals

The distinction between decided and propagated literals is crucial for backtracking:

- **Decided literals** (L^d): Chosen by the decision rule, can be flipped during backtracking
- **Propagated literals**: Forced by unit propagation, cannot be flipped (would immediately cause conflict)

The backtracking rule only flips decided literals, ensuring that propagated assignments remain consistent.

19 Termination and Results

The search process terminates when it reaches one of two possible outcomes:

19.1 Success State

Success Condition

Definition 19.1 (Satisfiable Instance)**.**

$$\langle S \parallel \Gamma \rangle \quad \text{returns} \quad (\text{"sat"}, \Gamma)$$

if and only if $\Gamma \models S$ (i.e., the assignment represented by trail Γ satisfies all clauses in S).

When a satisfying assignment is found, the solver returns:

- Status: `"sat"` (satisfiable)
- Model: The complete assignment Γ that satisfies the formula

19.2 Failure State

Failure Condition

Definition 19.2 (Unsatisfiable Instance)**.**

$$\langle S \parallel \Gamma \parallel C \rangle \quad \text{returns} \quad \text{"unsat"}$$

if and only if Γ contains no decision literals (i.e., all literals in the trail were propagated, not decided).

When the formula is proven unsatisfiable, the solver returns:

- Status: `"unsat"` (unsatisfiable)
- The empty clause $\square$ serves as a certificate of unsatisfiability

This occurs when the solver has exhausted all possible assignments without finding a satisfying one, meaning the formula is logically inconsistent.

20 Conflict-Driven Clause Learning (CDCL)

Modern SAT solvers use **Conflict-Driven Clause Learning (CDCL)**, an advanced technique that combines depth-first search with intelligent learning from conflicts to efficiently solve large satisfiability problems.

20.1 CDCL Algorithm Overview

CDCL implements a **depth-first search strategy** with intelligent backjumping and clause learning:

CDCL Components

CDCL combines several key techniques:

- **Unit propagation**: Efficient inference of forced assignments
- **Conflict analysis**: Learning from failures to guide future search
- **Backjumping**: Intelligent backtracking to earlier decision levels
- **Clause learning**: Adding learned constraints to prevent repeated conflicts

20.2 Backjumping: Generalized Backtracking

Backjumping is a generalization of the basic backtracking mechanism that allows the solver to jump back to earlier decision levels, not just the immediately preceding one.

Backjumping vs. Backtracking

Definition 20.1 (Backjumping)**.** In **backjumping**, when a conflict occurs, the solver analyzes the conflict clause to determine the earliest decision level that contributed to the conflict. It then jumps back to that level, rather than just the immediately preceding decision.
Advantage: Backjumping can skip over irrelevant decision levels, significantly reducing the search space compared to basic backtracking.

20.2.1 Conflict Analysis Example

Consider a conflict clause $C = L_1 \vee \cdots \vee L_k$ where all literals are currently false. The analysis determines:

- Why each literal L_i is false in the current trail Γ
- The decision level at which each literal became false
- The earliest decision level that contributed to the conflict

The solver then backjumps to that earliest relevant decision level, effectively skipping decision levels that did not contribute to the conflict.

21 Binary Resolution

The theoretical foundation of conflict analysis in CDCL relies on **binary resolution**, a fundamental inference rule in propositional logic.

21.1 Resolution Rule

Binary Resolution

Definition 21.1 (Binary Resolution)**.** Given two clauses containing complementary literals:

$$\frac{C \vee L \quad D \vee \neg L}{(C \vee D)}$$

where C and D are disjunctions of literals, and L is a literal.

The **resolvent** $C \vee D$ is logically entailed by the two premises. This rule allows deriving new clauses that preserve satisfiability.

21.2 Soundness of Resolution

Binary resolution is **sound**—the resolvent is logically entailed by the premises.

Soundness Proof

Theorem 21.1 (Resolution Soundness)**.** *If* $\{C \vee L, D \vee \neg L\} \models R$ *where* R *is any formula, then* $\{C \vee L, D \vee \neg L\} \models (C \vee D)$.

Proof. Consider any interpretation v:

- If $v \models L$, then since $v \models D \vee \neg L$, we must have $v \models D$. Since $v \models C \vee L$, we have $v \models C \vee D$.
- If $v \models \neg L$, then since $v \models C \vee L$, we must have $v \models C$. Since $v \models D \vee \neg L$, we have $v \models C \vee D$.

In both cases, $v \models C \vee D$, so the resolvent is logically entailed by the premises. □

21.3 Application in Conflict Analysis

In conflict analysis, resolution is used to explain why conflicts occur and to determine which decision levels are responsible:

- When a literal L_k in the conflict clause is false due to propagation (not a decision)
- The propagation was justified by a clause $D \vee \neg L_k$
- By resolving the conflict clause with the justification clause, we can trace back the decision that ultimately caused the conflict

22 Explanation Mechanism

The explanation mechanism in CDCL traces the implications that led to a conflict, allowing the solver to learn more general constraints.

Explanation Transition

Definition 22.1 (Explanation Rule)**.**

$$\langle S \parallel \Gamma \parallel C \rangle \to \langle S \parallel \Gamma \parallel C \vee D \rangle$$

if $\neg L \in \Gamma$ where $\neg L$ was propagated using the clause $D \vee \neg L$.

22.1 Explanation Process

The explanation mechanism works as follows:

1. Start with a conflict clause $C = L_1 \vee \cdots \vee L_k$
2. For each literal L_i that is false due to propagation (not a decision):
   - Find the justification clause that forced $\neg L_i$ into the trail
   - Resolve the current conflict clause with the justification clause
   - Continue until all literals in the conflict clause are explained by decisions
3. The final resolved clause contains only literals that were decided (not propagated)
4. This clause can be learned and added to the formula to prevent similar conflicts

Part VI

SAT-Based Techniques

23 Modern SAT Solving

23.1 CDCL: An Evolution of DPLL

CDCL (**Conflict-Driven Clause Learning**) represents an evolution of the DPLL decision procedure for SAT. While CDCL maintains the same basic decision rule as DPLL, it introduces sophisticated conflict analysis and learning mechanisms that dramatically improve solver efficiency on real-world instances.

23.1.1 CDCL Transition Rules

The CDCL procedure employs the following transition rules in search mode:

Decision Rule

Definition 23.1 (Decision)**.**

$$\langle S \parallel \Gamma \rangle \to \langle S \parallel \Gamma, L^d \rangle$$

if $L \notin \Gamma$ and $\neg L \notin \Gamma$, where:

- S is the set of clauses
- Γ is the trail (sequence of assigned literals)
- L^d is a decision literal (chosen by a heuristic)

Propagation Rule

Definition 23.2 (Propagation)**.**

$$\langle S \parallel \Gamma \rangle \to \langle S \parallel \Gamma, L_{C \vee L} \rangle$$

if $C \vee L \in S$, $L \notin \Gamma$, $\neg L \notin \Gamma$, and $\Gamma \models \neg C$, where:

- C is a disjunction of literals (all false under the current trail)
- L is the unit literal that must be propagated
- The subscript $C \vee L$ indicates the justification clause for the propagation

Success Rule

Definition 23.3 (Success)**.**

$$\langle S \parallel \Gamma \rangle \to \text{"sat"}$$

if $\Gamma \models S$ (i.e., the trail satisfies all clauses in the set)

Conflict Rule

Definition 23.4 (Conflict)**.**

$$\langle S \parallel \Gamma \rangle \to \langle S \parallel \Gamma \parallel C \rangle$$

if $C \in S$ and $\Gamma \models \neg C$ (i.e., the trail conflicts with clause C), where C is the conflict clause that triggers the transition to conflict analysis mode.

These four rules define the behavior of CDCL in **search mode**, which is identical to the DPLL

procedure. The key difference lies in how CDCL handles conflicts through its conflict analysis and learning mechanisms.

23.2 Decision Levels in CDCL

The trail Γ in CDCL is organized into **decision levels** that provide structure to the search process:

- The trail begins at **decision level 0** (empty trail)
- Each decision literal opens a new decision level
- If the maximum decision level in Γ is n, the next decision opens level $n + 1$
- All propagations following a decision remain at the same level until the next decision

23.2.1 Trail as a Stack Structure

The trail can be conceptualized as a **stack structure**:

1. Initially, the stack is empty (decision level 0)
2. A decision literal is pushed onto the stack, opening a new level
3. Unit propagations following the decision are pushed onto the stack at the same level
4. Another decision is made, opening the next level
5. The process continues with alternating decisions and propagations
6. During backtracking/backjumping, literals are popped from the stack until reaching the target decision level

This organization enables efficient backjumping by allowing the solver to quickly identify and return to relevant decision levels during conflict analysis.

23.3 Illustrative Example

Instance

$S = \{ P,\ \neg P \vee Q \vee R,\ T \vee \neg Q,\ \neg R \vee Q \}$

The initial propagation and decision sequence proceeds as follows:

$$\begin{aligned} \langle S \parallel \epsilon \rangle &\xrightarrow{\text{prop}} \langle S \parallel P_P \rangle \\ &\xrightarrow{\text{dec}} \langle S \parallel P_P,\ Q^d \rangle \\ &\xrightarrow{\text{prop}} \langle S \parallel P_P,\ Q^d,\ T_{T \vee \neg Q} \rangle \end{aligned}$$

Remark. The trail behaves exactly like a *stack*: each decision opens a new level and subsequent propagations are pushed on top of that level.

23.4 Conflict Mode

When a conflict clause C becomes false under the current trail, the solver enters conflict mode. Two outcomes are possible:

1. **Fail**: The formula is unsatisfiable.

2. **Conflict resolution**: The solver analyzes the conflict, learns a new clause, backjumps, and resumes search.

23.4.1 Fail Rule

Fail Condition

Definition 23.5 (Unsat at Level 0)**.**

$$\langle S \parallel \Gamma \parallel C \rangle \;\to\; \text{"unsat"}$$

if $\max\text{level}(\Gamma) = 0$.

23.4.2 Explanation Rule

Assume the conflict clause has the form

$$C = L_1 \vee L_2 \vee \cdots \vee L_{k-1} \vee L_k, \quad \text{all literals false in } \Gamma.$$

If $\neg L_k$ was propagated by the clause $D \vee \neg L_k$, we resolve C with this justification to obtain a new clause:

$$C' = L_1 \vee L_2 \vee \cdots \vee L_{k-1} \vee D.$$

Explanation Transition

Definition 23.6.

$$\langle S \parallel \Gamma \parallel C \vee L \rangle \;\to\; \langle S \parallel \Gamma \parallel C \vee D \rangle$$

if $\neg L_{D \vee \neg L} \in \Gamma$.

The resolution step is repeated until the resulting clause contains exactly one literal from the current decision level (*1-UIP clause*).

23.4.3 Learning Rule

Clause Learning

Definition 23.7.

$$\langle S \parallel \Gamma \parallel C \rangle \;\to\; \langle S \cup \{C\} \parallel \Gamma \parallel C \rangle$$

if $C \notin S$.

The learned clause C prevents the solver from repeating the same conflicting assignment in the future.

23.4.4 Backjumping Rule

Backjumping

Definition 23.8. Let $\Gamma = N\, L^d\, M$ where L^d is the most recent decision literal and $C \vee L'$ is the learned clause with L' falsified at level $k < \text{level}(L^d)$. Then

$$\langle S \parallel N\, L^d\, M \parallel C \vee L' \rangle \;\to\; \langle S \parallel N \cup \{L'_{C \vee L'}\} \rangle.$$

Backjumping skips irrelevant decision levels, jumping directly to level k, and immediately propagates L' using the learned clause.

23.5 Control Strategy

The transition rules of CDCL are governed by specific control policies that determine the order in which rules are applied.

23.5.1 Search Mode Control

Search Mode Priority

In search mode, **Propagation** has higher priority than **Decision**.

This ensures that all forced assignments (unit propagations) are made before any heuristic decision is taken, maximizing constraint propagation and reducing the search space.

23.5.2 Conflict Mode Control

Conflict Mode Sequence

The conflict mode rules are applied in the following order:

$$\text{Explanation}^* \rightarrow \text{Learning} \rightarrow \text{Backjumping}$$

where Explanation* denotes repeated application of the Explanation rule.

23.6 First Assertion Clause Heuristic

The **First Assertion Clause (FAC)** heuristic is a widely used strategy for determining when to stop the explanation process and perform backjumping.

Assertion Clause

Definition 23.9 (Assertion Clause)**.** An **assertion clause** is a conflict clause $C \vee L'$ such that exactly one of its literals (the **assertion literal** L') is made false by the current (highest) decision level in Γ, while all other literals are false at earlier decision levels.

23.6.1 FAC Heuristic Strategy

The First Assertion Clause heuristic operates as follows:

1. Apply the **Explanation** rule repeatedly until an assertion clause is generated
2. **Learn** the assertion clause by adding it to the clause set S
3. **Backjump** to the smallest decision level where:
   - The assertion literal L' is undefined
   - All other literals in the assertion clause are still false
4. Exit conflict mode by placing the assertion literal L' on the trail with the assertion clause as its justification

24 Exercise: CDCL vs. Chronological Backtracking

Exercise 24.1. Consider the propositional clause set

$$S = \{\, \neg 1 \vee 2,\ \neg 3 \vee 4,\ \neg 5 \vee \neg 6,\ 6 \vee \neg 5 \vee \neg 2 \,\}.$$

Perform the following tasks.

1. **Explore the search tree.** Starting from an empty trail, show every propagation and decision (in DPLL style) until the first conflict is encountered. Record the decision level of each literal.
2. **Chronological backtracking.** Resolve the conflict by undoing only the last decision. Explain why the solver believes the conflict is fixed even though the same assignment can re-appear later.
3. **CDCL with First Assertion Clause (FAC).** Handle the same conflict using CDCL:
   (a) Apply the *explanation* rule repeatedly until the first assertion clause (1-UIP clause) is obtained.
   (b) *Learn* this clause by adding it to S.
   (c) Determine the backjump level implied by the clause and show the exact trail after backjumping and the immediate unit propagation.
4. **Compare the two approaches.** Discuss why the learned clause prevents the solver from revisiting the conflicting assignment and how this prunes the search space more effectively than chronological backtracking.

24.0.1 Understanding the Clause Set

Before solving, let us interpret the logical constraints encoded in S:

- $C_1 = \neg 1 \vee 2$: If literal 1 is true, then literal 2 must also be true
- $C_2 = \neg 3 \vee 4$: If literal 3 is true, then literal 4 must also be true
- $C_3 = \neg 5 \vee \neg 6$: Literals 5 and 6 cannot both be true simultaneously
- $C_4 = 6 \vee \neg 5 \vee \neg 2$: At least one must hold: 6 is true, OR 5 is false, OR 2 is false

Example 24.1 (Detailed Solution)**.** Throughout the solution, literals annotated with ^{d} are decisions; subscripts indicate the clause that justified a propagation. Decision levels are shown on the left margin.

1. Exploring the Search Tree Until Conflict Starting Configuration (Level 0): The solver begins with an empty trail:

$$\langle S \parallel \epsilon \rangle$$

Decision Level 1: The solver makes its first decision, setting literal 1 to true:

$$\xrightarrow{\text{dec}} \langle S \parallel 1^{d} \rangle$$

This immediately triggers unit propagation. Since 1 is true, the literal $\neg 1$ in clause $C_1 = \neg 1 \vee 2$ becomes false, making 2 the only literal that can satisfy the clause. Therefore, 2 must be propagated:

$$\xrightarrow{\text{prop}} \langle S \parallel 1^{d},\ 2_{\neg 1 \vee 2} \rangle$$

Decision Level 2: With no further propagations possible, the solver decides to set literal 3 to true:

$$\xrightarrow{\text{dec}} \langle S \parallel 1^d, 2, 3^d \rangle$$

By the same reasoning, clause $C_2 = \neg 3 \vee 4$ forces the propagation of literal 4:

$$\xrightarrow{\text{prop}} \langle S \parallel 1^d, 2, 3^d, 4_{\neg 3 \vee 4} \rangle$$

Decision Level 3: The solver makes another decision, setting literal 5 to true:

$$\xrightarrow{\text{dec}} \langle S \parallel 1^d, 2, 3^d, 4, 5^d \rangle$$

Clause $C_3 = \neg 5 \vee \neg 6$ now has $\neg 5$ false (since 5 is true), so $\neg 6$ must be propagated:

$$\xrightarrow{\text{prop}} \langle S \parallel 1^d, 2, 3^d, 4, 5^d, \neg 6_{\neg 5 \vee \neg 6} \rangle$$

Conflict Detection: Examining clause $C_4 = 6 \vee \neg 5 \vee \neg 2$, we find all three literals are false:

- 6 is false (because $\neg 6$ is on the trail at level 3)
- $\neg 5$ is false (because 5 is true on the trail at level 3)
- $\neg 2$ is false (because 2 is true on the trail at level 1)

The clause is violated, triggering a conflict at level 3.

2. Resolving the Conflict: Chronological Backtracking The Classical DPLL Approach: Chronological backtracking simply undoes the most recent decision. The solver performs the following steps:

1. Remove decision 5^d and all its consequences ($\neg 6$) from the trail
2. Return to decision level 2 with trail:

$$\langle S \parallel 1^d, 2, 3^d, 4 \rangle$$

3. Try the alternative assignment $\neg 5$ and continue searching

Why the Conflict Appears Resolved: After removing 5^d and $\neg 6$ from the trail, clause C_4 is no longer violated. The literals are now:

- 6 is undefined (not on the trail)
- $\neg 5$ is undefined
- $\neg 2$ is still false (but C_4 is satisfied by undefined literals)

The solver can resume search without an immediate conflict.

The Fatal Flaw: The clause set S still does not prevent the combination of 2 and 5 both being true. If the search explores a different path that again assigns both 2 = true and 5 = true, the exact same conflict will occur. The solver has learned nothing permanent and may waste significant time re-exploring this futile combination multiple times in different parts of the search tree.

3. CDCL with First Assertion Clause (a) Conflict Analysis: CDCL analyzes the conflict clause $C_4 = 6 \vee \neg 5 \vee \neg 2$ to understand its root cause. The clause contains three false literals:

- 6 is false because $\neg 6$ was *propagated* at level 3 (not decided)
- $\neg 5$ is false because 5 was *decided* at level 3
- $\neg 2$ is false because 2 was *propagated* at level 1

(b) Resolution and Explanation: The literal $\neg 6$ was not a decision—it was propagated using clause $C_3 = \neg 5 \vee \neg 6$. To trace back the cause of the conflict, we resolve the conflict clause with its justification clause:

$$\frac{C_4 = (6 \vee \neg 5 \vee \neg 2) \qquad C_3 = (\neg 5 \vee \neg 6)}{C_{\text{learned}} = \neg 5 \vee \neg 2}$$

The resolvent $C_{\text{learned}} = \neg 5 \vee \neg 2$ is an **assertion clause** because:

- Exactly one literal ($\neg 5$) is false at the current decision level 3 (the assertion literal)
- The other literal ($\neg 2$) is false at an earlier level (level 1)

(c) Learning: The solver adds the learned clause to the clause set:

$$S \leftarrow S \cup \{\neg 5 \vee \neg 2\}$$

This new clause permanently encodes the constraint: *literals* 2 *and* 5 *cannot both be true.*

(d) Backjumping: Instead of returning to level 2, CDCL identifies that the conflict was caused by the interaction between:

- Level 1 (where 2 was propagated)
- Level 3 (where 5 was decided)

Decision level 2 (the decision 3^d and propagation of 4) played *no role* in the conflict.

Therefore, the solver **backjumps** directly to level 1, skipping level 2 entirely:

$$\langle S \cup \{\neg 5 \vee \neg 2\} \parallel 1^d, 2 \rangle$$

(e) Unit Propagation from Learned Clause: At level 1, literal 2 is true, making $\neg 2$ false. The learned clause $\neg 5 \vee \neg 2$ now becomes a unit clause, forcing the immediate propagation:

$$\langle S \cup \{\neg 5 \vee \neg 2\} \parallel 1^d, 2, \neg 5_{\neg 5 \vee \neg 2} \rangle$$

The solver immediately deduces that 5 must be false whenever 2 is true, preventing the conflict from ever occurring again.

4. Comparative Analysis Summary of Key Differences:

Aspect	**Chronological Backtracking**	**CDCL**
Backtrack level	Returns to level 2	Jumps to level 1
Knowledge gained	None—can repeat same conflict	Learns $\neg 5 \vee \neg 2$ permanently
Search space	May re-explore conflicting assignments	Prunes entire subtrees where $2 \wedge 5$ holds
Efficiency	Linear exploration with repetition	Exponential pruning through learning

Why the Learned Clause Is Powerful:

The clause $\neg 5 \vee \neg 2$ acts as a **global constraint** that permanently blocks the combination $(2, 5)$ throughout the entire search:

- Any future branch that tries to set both 2 = true and 5 = true will violate the clause immediately
- The violation triggers unit propagation, forcing one of the literals to be false before deeper exploration
- This prevents the solver from wasting time on futile search paths

CDCL therefore prunes an *entire subtree* of the search space in a single learning step, while chronological backtracking may explore that subtree repeatedly in different parts of the search tree.

Fundamental Advantages of CDCL:

1. **Non-chronological backjumping**: By analyzing conflict clauses, CDCL can identify and jump directly to the earliest decision level that contributed to the conflict, skipping irrelevant intermediate levels.
2. **Clause learning**: Each conflict generates a new constraint (learned clause) that captures the reason for the failure. This clause remains in the database permanently, preventing the same mistake from happening again.
3. **Search space pruning**: Learned clauses effectively prune exponentially large portions of the search space. A single learned clause can eliminate millions or billions of invalid variable assignments.
4. **Guided search**: The learned clauses provide additional unit propagations that guide the solver away from unproductive regions of the search space, focusing effort on more promising areas.

These mechanisms make CDCL dramatically more efficient than classical DPLL on real-world SAT instances, enabling modern solvers to handle formulas with millions of variables and clauses.

25 CDCL with Proof Generation

25.1 Motivation for Proof Generation

When we submit a formula F to a SAT solver and receive an `unsat` verdict, in many applications a simple yes/no answer is insufficient. For unsatisfiable formulas, we often need a **proof of unsatisfiability** that can be independently verified.

25.1.1 Resolution Proofs

The resolution rule is the foundation for unsatisfiability proofs:

Resolution Rule

$$\frac{C \vee L \quad D \vee \neg L}{C \vee D}$$

From two clauses containing complementary literals L and $\neg L$, we can derive the resolvent $C \vee D$.

A **resolution proof** is represented as a resolution proof tree with the following structure:

- The **root** is labeled by the empty clause $\square$
- The **leaves** are labeled by input clauses
- The **intermediate nodes** are labeled by resolvents other than $\square$

Example 25.1 (Simple Unsatisfiable Formula)**.** Consider the easily unsatisfiable set of clauses:

$$S = \{P \vee Q, \neg P \vee Q, P \vee \neg Q, \neg P \vee \neg Q\}$$

Through successive resolution steps, this reduces to the empty clause $\square$, proving unsatisfiability.

25.2 Generating Resolution Proofs in CDCL

The key question is: how do we generate resolution proofs inside a CDCL procedure?

CDCL Proof Generation

Without proof generation:

$$\text{Fail: } S \parallel \Gamma \parallel C \longrightarrow \texttt{"unsat"}$$

if the only level in trail Γ is level 0.

With proof generation:

$$\text{Fail: } S \parallel \Gamma \parallel \square \longrightarrow \texttt{"unsat"}$$

Perform additional Explanation steps after reaching a conflict state where the only level on the trail is level 0.

Proof: All the resolution inference steps (encapsulated in Explanation transitions) that connect $\square$ back to the input clauses form the resolution proof.

Example 25.2 (CDCL Execution with Proof Generation)**.** Consider the formula:

$$F = \{p \vee q,\ \neg p \vee q,\ p \vee \neg q,\ s \vee \neg p \vee q,\ \neg s \vee p \vee \neg q,\ \neg p \vee r,\ \neg q \vee \neg r\}$$

The CDCL execution trace with proof generation:

$$
\begin{aligned}
F \parallel \epsilon &\xrightarrow{\text{Decision}} F \parallel s^d \\
&\xrightarrow{\text{Decision}} F \parallel s^d r^d \\
&\xrightarrow{\text{Propagation}} F \parallel s^d r^d \neg q \\
&\xrightarrow{\text{Propagation}} F \parallel s^d r^d \neg q \neg p \\
&\xrightarrow{\text{Conflict}} F \parallel s^d r^d \neg q \neg p \parallel p \vee q \\
&\xrightarrow{\text{Explanation}} F \parallel s^d r^d \neg q \neg p \\
&\xrightarrow{\text{Learning}} F \cup \{q\} \parallel s^d r^d \neg q \neg p \parallel q \\
&\xrightarrow{\text{Backjumping}} F \cup \{q\} \parallel q \\
&\xrightarrow{\text{Propagation}} F \cup \{q\} \parallel q\ p \\
&\xrightarrow{\text{Propagation}} F \cup \{q\} \parallel q\ p\ r \\
&\xrightarrow{\text{Conflict}} F \cup \{q\} \parallel q\ p\ r \parallel \neg q \vee \neg r \\
&\xrightarrow{\text{Explanation}} F \cup \{q\} \parallel q\ p\ r \parallel \neg q \vee \neg p \\
&\xrightarrow{\text{Explanation}} F \cup \{q\} \parallel q\ p\ r \parallel \neg q \\
&\xrightarrow{\text{Explanation}} F \cup \{q\} \parallel q\ p\ r \parallel \square \\
&\xrightarrow{\text{Fail}} \texttt{"unsat"}
\end{aligned}
$$

The proof is represented as a resolution tree showing how $\square$ was derived from the input clauses.

25.3 Decision Heuristics in CDCL

The choice of which variable to branch on (and which truth value to assign first) can dramatically affect solver performance. Modern CDCL solvers employ sophisticated decision heuristics.

25.3.1 First Fail Principle

The **First Fail Principle** suggests that we should try to detect failures as early as possible in the search. This leads to two main strategies:

First Fail Heuristics

Strategy I (Literal-based):
Pick a literal with the highest number of occurrences in the current clause set (i.e., input clauses + learned clauses). Ties are broken arbitrarily.

Strategy II (Variable-based):
Pick a variable with the maximum number of occurrences in the current set, then pick the sign (polarity) with the most occurrences. Ties among variables are broken arbitrarily.

Optimization: Apply Strategy I or II only to clauses of length up to k for some parameter k, as short clauses are more likely to become unit clauses or conflict clauses.

25.3.2 VSIDS (Variable State Independent Decaying Sum)

VSIDS is one of the most successful decision heuristics in modern SAT solvers. The core idea is to **measure variable activity** based on participation in conflicts.

VSIDS Algorithm

Data structure: Keep a counter (activity score) for every variable.

Initialization:
- Set to 0 if we ignore the input formula
- Or set to the number of occurrences of the variable in the input clause set

Update rule: Bump (increment) the counter whenever the variable appears in a learned clause.

Decay mechanism: Periodically reduce all counters to give more weight to recent activity:
- Subtract k (where $k \geq 0$), or
- Divide by k (where $k > 1$), or
- Augment the increment so that more recent activity weighs more

Decision: Choose the variable with the highest activity score.

Variant - EVSIDS (Extended VSIDS): Instead of counting only variables in learned clauses, count variables in *all* resolution steps performed during conflict analysis, not just the final learned clause.

25.4 Implementation of Propagation: The 2-Watched Literal Scheme

The 2-watched literal scheme is an efficient data structure for implementing Boolean Constraint Propagation (BCP) in CDCL solvers.

Goals of the 2-Watched Literal Scheme

- Discover implied literals efficiently
- Detect conflict clauses quickly

25.4.1 Core Invariant

For every clause, maintain **2 watched literals** that are not false (i.e., either true or undefined). This invariant is maintained throughout the search.

25.4.2 Update Rules

The scheme operates according to the following rules when literals are assigned:

1. **Case 1 - One watched literal becomes false:**

   If L_1 becomes false and L_2 remains non-false, look for another literal L_3 that is non-false:

   - If we find L_3: set L_2 and L_3 as the new pair of watched literals
   - If we cannot find L_3: then L_2 is implied (it must be true to satisfy the clause)

2. **Case 2 - Both watched literals become false:**

   If both L_1 and L_2 become false, look for two replacement literals L_3 and L_4:

   - If we find both L_3 and L_4: set them as the new pair of watched literals
   - If we find only one (say L_3): then L_3 is implied
   - If we find none: the clause is a **conflict clause**

3. **Case 3 - Otherwise:**

   If neither watched literal becomes false, do nothing (no update needed).

Efficiency: This scheme is highly efficient because it requires examining only the two watched literals per clause during propagation, rather than scanning all literals in every clause. The watched literals are updated lazily only when necessary.

26 Advanced CDCL Techniques

26.1 Clause Forgetting and Restart Strategies

We complete the discussion of the CDCL procedure for SAT by introducing two important optimizations that improve practical performance: **clause forgetting** and **restart strategies**.

While CDCL has the capacity to learn clauses that help prune the search space, unbounded clause learning can lead to memory problems and decreased efficiency. Modern SAT solvers address this through selective clause deletion.

26.1.1 The Forget Transition Rule

The **Forget** rule allows the solver to remove learned clauses that are no longer deemed useful, preventing the clause database from growing indefinitely.

Forget Transition Rule

Let S_0 denote the input clause set. The Forget rule is defined as:

$$\text{Forget: } S \parallel \mathcal{M} \longrightarrow S \setminus \{C\} \parallel \mathcal{M}$$

if $C \in S$ and $C \notin S_0$ (i.e., C is a learned clause).

Which clauses should we forget?

Heuristic criteria for clause deletion typically include:

- **Clause length criterion**: Forget clauses where $|C| > k$ for some integer parameter k, where $|C|$ denotes the number of literals in C. Short learned clauses tend to be more useful for propagation.
- **Periodic deletion**: Apply the forgetting mechanism only when the number of transition steps performed in the derivation is a multiple of some period T. This prevents excessive overhead from constant clause management.
- **Activity-based deletion**: Track clause activity (how often a clause participates in conflict analysis or propagation) and preferentially retain active clauses.

26.1.2 The Restart Transition Rule

The **Restart** rule allows the solver to abandon the current search path and restart from scratch, while keeping all learned clauses. This helps escape unproductive regions of the search space.

Restart Transition Rule

$$\text{Restart: } S \parallel \mathcal{M} \longrightarrow S \parallel \mathcal{M}_0$$

where $\mathcal{M}_0$ is the prefix of $\mathcal{M}$ containing all and only the literals at level 0 in $\mathcal{M}$.
Important: All learned clauses in S are kept across restarts.

Rationale: While restarting discards the current partial assignment, the learned clauses accumulated so far guide the search away from previously explored unpromising regions. This often leads to finding solutions or proofs much faster than exhaustively exploring a single search path.

26.2 First Assertion Clause Strategy

We use the **First Assertion Clause** heuristic for determining the backjump level:

Backjump with First Assertion Clause

$$S \parallel \mathcal{M}\ L^d\ \mathcal{N} \parallel C \vee L' \longrightarrow S \parallel \mathcal{M}\ L'_{C \vee L'}$$

if $C \vee L' \in S$ and $\mathcal{M} \models \neg C$, where L' is the **assertion literal** (the only literal in $C \vee L'$ which is falsified by the current decision level).

26.2.1 Why Choose the First Assertion Clause?

Suppose we have a conflict clause:

$$L_1^{n_1} \vee L_2^{n_2} \vee \cdots \vee L_k^{n_k} \vee L_{k+1}^{n}$$

where:

- n is the current decision level
- n_i is the level where literal L_i is falsified
- $n > n_i$ for all $i \in \{1, \ldots, k\}$
- L_{k+1}^n is the assertion literal at level n

This is a **first assertion clause** (also called a **1-UIP clause**) because exactly one literal is falsified at the current level.

26.2.2 Resolution and Backjump Levels

Consider the resolution inference:

$$\frac{\neg L \vee C \qquad L \vee D}{C \vee D}$$

Let:

- p = maximum level that falsifies literals in C
- q = maximum level that falsifies literals in $L \vee D$
- ℓ = maximum level that falsifies literals in $C \vee D$

Then:

$$\ell = \max(p, q)$$

Key insight for backjumping: When we backjump, we need to ensure that the level we return to still falsifies all literals in the learned clause except the assertion literal. The longer the learned clause (i.e., the more literals it contains at different levels), the shorter the backjump may be. Conversely, using the first assertion clause (which has only one literal at the current level) allows for maximal backjumping, potentially skipping many irrelevant decision levels.

27 Temporal Planning and Constraint Solving

27.1 Extensions to Classical Planning

Beyond the classical planning framework, there exist several important extensions that allow for more expressive problem formulations. One key extension involves **temporal reasoning**, which allows us to reason about the duration of actions and temporal relationships between events.

27.1.1 Temporal Constraints

Temporal planning introduces constraints about **time**, specifying when certain actions should occur and how they relate temporally.

Simple temporal constraints:

$$t < t'$$

where t and t' are time points at which certain actions occur.

Duration constraints:

$$d \leq t' - t \leq d'$$

which can be equivalently written as:

$$(d \leq t' - t) \wedge (t' - t \leq d')$$

Here:

- d is the **lower bound** for the duration
- d' is the **upper bound** for the duration
- $t' - t$ represents the duration of the time interval between two actions performed at times t and t', respectively

These constraints allow us to model requirements such as "action A must complete at least 5 time units before action B starts" or "the total duration of action C must be between 3 and 7 time units."

27.2 Constraint Solving in Arithmetic

To solve temporal planning problems, we need to address the question: **How do we solve Constraint Satisfaction Problems (CSP) in arithmetic?**

Arithmetic constraints can be classified into two main categories:

- **Linear arithmetic**: Involves only addition and subtraction (algebraic sums). Products are allowed only between a constant c and a variable x, but *not* between variables.
- **Non-linear arithmetic**: Allows products between variables and other non-linear operations.

27.2.1 Decidability of Arithmetic Theories

Different arithmetic theories have different computational properties:

Theory	**Decidability**
LIA (Linear Integer Arithmetic)	Decidable
NIA (Non-linear Integer Arithmetic)	Undecidable
LRA (Linear Rational/Real Arithmetic)	Decidable
NRA (Non-linear Real Arithmetic)	Decidable

For temporal planning, we focus on **LRA (Linear Rational/Real Arithmetic)** because it is decidable and sufficiently expressive for modeling temporal constraints.

27.3 Linear Rational Arithmetic (LRA)

In LRA, we consider a set of constraints of the form:

$$L \bowtie R$$

where $\bowtie \in \{<, \leq, =\}$.

We distinguish between:

- **Strict inequality**: $<$
- **Weak inequality**: $\leq$
- **Equality**: $=$

Structure of constraints:

Both L and R are algebraic sums of linear monomials:

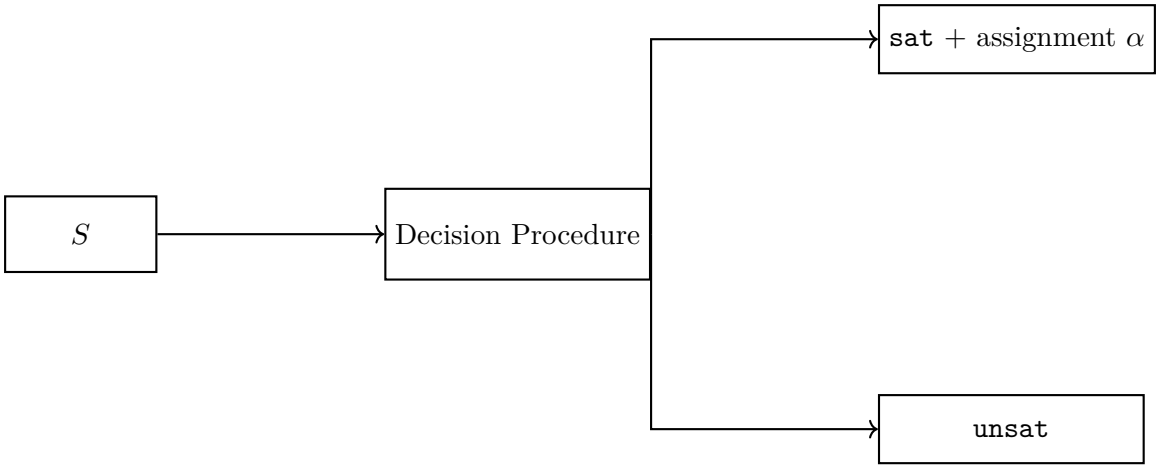
$$c_1 \cdot x_1 + c_2 \cdot x_2 + \cdots + c_k \cdot x_k$$

where:

- $x_1, x_2, \ldots, x_k$ are **variables**
- $c_1, c_2, \ldots, c_k$ are **constants** (rational numbers)

27.3.1 Decision Procedure for LRA

Given a set S of LRA constraints, a decision procedure determines:



Example 27.1 (LRA Constraint System)**.** Consider the following system of constraints:

$$\begin{aligned} x + y &\geq 2 \\ 2x - y &\geq 0 \\ -x + 2y &\geq 1 \end{aligned}$$

These are typical LRA constraints we want to solve.

27.4 The Simplex Method

There are two variants of the Simplex method relevant to our discussion:

Classical Simplex Method

Solves **optimization problems**: Given a set S of LRA constraints and an objective function f over variables in S, find an assignment $\alpha : X \to \mathbb{Q}$ that satisfies S and maximizes (or minimizes) f.

However, for constraint satisfaction we do **not** need optimization. Instead, we use:

General Simplex Method

Solves **CSP in LRA**: Given a set S of LRA constraints, find an assignment $\alpha : X \to \mathbb{Q}$ that satisfies S (without optimization).

27.4.1 General Form Transformation

Step I: Transform the input problem into general form

Introduce **slack variables** to convert inequalities into equalities plus non-negativity constraints:

Example 27.2 (Slack Variable Introduction)**.** Starting from the constraint system:

$$\begin{aligned} x + y &\geq 2 \\ 2x - y &\geq 0 \\ -x + 2y &\geq 1 \end{aligned}$$

We introduce slack variables s_1, s_2, s_3 and rewrite as:

$$\begin{aligned} x + y - s_1 = 0 \quad &\wedge \quad s_1 \geq 2 \\ 2x - y - s_2 = 0 \quad &\wedge \quad s_2 \geq 0 \\ -x + 2y - s_3 = 0 \quad &\wedge \quad s_3 \geq 1 \end{aligned}$$

This transformation separates the structural equations (equalities) from the bound constraints (inequalities on slack variables).

The general form consists of:

- A system of linear equations (the structural equations with slack variables)
- Bound constraints on the slack variables

This standard form is the basis for applying the Simplex algorithm to determine satisfiability of the constraint system.

27.5 The General Simplex Algorithm

Having discussed Step I (transformation into general form), we now present **Step II**: the core algorithm for satisfiability checking.

27.5.1 Detailed Transformation Process (Step I Recap)

For each input constraint $L \bowtie R$ where $\bowtie \in \{\leq, <, =\}$:

1. Rewrite it into the form $L' \bowtie b$ where L' is an LRA-term and b is a constant
2. Introduce an additional **slack variable** s and rewrite:

$$L' \bowtie b \quad \Rightarrow \quad L' - s = 0 \wedge s \bowtie b$$

3. If $\bowtie$ is $=$, rewrite the bound constraint as:

$$s = b \quad \Rightarrow \quad s \leq b \wedge s \geq b$$

Example 27.3 (Complete Transformation)**.** Starting from:

$$\begin{aligned} x + y &\geq 2 \\ 2x - y &\geq 0 \\ -x + 2y &\geq 1 \end{aligned}$$

After introducing slack variables s_1, s_2, s_3:

$$\begin{aligned} x + y - s_1 = 0 \quad &\wedge \quad s_1 \geq 2 \\ 2x - y - s_2 = 0 \quad &\wedge \quad s_2 \geq 0 \\ -x + 2y - s_3 = 0 \quad &\wedge \quad s_3 \geq 1 \end{aligned}$$

Variables:

- **Problem variables**: $\{x, y\}$ (original variables)
- **Slack variables**: $\{s_1, s_2, s_3\}$ (additional variables)

27.5.2 Tableau Representation

The transformation creates a matrix A representing the system $Ax = 0$. This matrix is formed by:

- Coefficients of the problem variables $x_1, \ldots, x_n$
- A diagonal submatrix for the slack variables $s_1, \ldots, s_m$ (with -1 on the diagonal)

Key concepts:

- $\mathcal{B}$ = set of **basic variables** (dependent variables)
- $\mathcal{N}$ = set of **non-basic variables** (independent variables)
- $\alpha : X \to \mathbb{Q}$ = assignment function where $X = \{x_1, \ldots, x_n, s_1, \ldots, s_m\}$

27.5.3 Step II: The Simplex Algorithm for Satisfiability

General Simplex Algorithm

Initialization:

- $\mathcal{B} = \{s_1, \ldots, s_m\}$ (basic variables are initially the slack variables)
- $\mathcal{N} = \{x_1, \ldots, x_n\}$ (non-basic variables are the problem variables)
- $\alpha(x) = 0$ for all $x \in \mathcal{B} \cup \mathcal{N}$

Invariants maintained throughout:

- **Inv-1**: $Ax = 0$ is satisfied by α (structural equations hold)
- **Inv-2**: The lower and upper bounds on variables in $\mathcal{N}$ are satisfied

Algorithm:

Construct the initial tableau. Let $<$ be a fixed ordering on $\mathcal{B} \cup \mathcal{N}$.

while `true` **do**
  if α satisfies the bounds on all basic variables **then**
    return $(\texttt{sat}, \alpha)$
  end if
  Let x_i be the $<$-smallest basic variable such that $\alpha(x_i)$ violates its bounds
  Look for a suitable non-basic variable x_j
  if no suitable x_j exists **then**
    return $(\texttt{unsat}, \bot)$
  else
    Let x_j be the $<$-smallest suitable non-basic variable
    Perform **pivoting**: update α and swap x_i and x_j between $\mathcal{B}$ and $\mathcal{N}$
  end if
end while

27.5.4 Pivoting: Finding Suitable Non-Basic Variables

Suppose basic variable x_i violates its bounds. We have x_i expressed in terms of non-basic variables:

$$x_i = \sum_{j=1}^{|\mathcal{N}|} a_{ij} x_j$$

Case 1: $\alpha(x_i) < l_i$ (current value is below the lower bound)

We need to **augment** $\alpha(x_i)$. A non-basic variable x_j is **suitable** if:

- $a_{ij} > 0$ and $\alpha(x_j) < u_j$ (we can increase x_j), **or**
- $a_{ij} < 0$ and $\alpha(x_j) > l_j$ (we can decrease x_j)

Case 2: $\alpha(x_i) > u_i$ (current value is above the upper bound)

We need to **reduce** $\alpha(x_i)$. A non-basic variable x_j is **suitable** if:

- $a_{ij} > 0$ and $\alpha(x_j) > l_j$ (we can decrease x_j), **or**
- $a_{ij} < 0$ and $\alpha(x_j) < u_j$ (we can increase x_j)

27.5.5 Update Rule

Once a suitable x_j is found, we compute the change Δ and update the assignment:

$$\Delta = \frac{\text{target value} - \alpha(x_i)}{a_{ij}}$$

For example, if $\alpha(x_i) < l_i$, we set:

$$\Delta = \frac{l_i - \alpha(x_i)}{a_{ij}}, \quad \alpha(x_j) := \alpha(x_j) + \Delta$$

Then we update $\alpha(x_i)$ and all other basic variables that depend on x_j accordingly, and swap x_i and x_j between $\mathcal{B}$ and $\mathcal{N}$.

Remark. The algorithm maintains the invariants Inv-1 and Inv-2 at each step. Termination is guaranteed because:

- If we find a satisfying assignment, we return `sat`
- If we cannot find a suitable non-basic variable to fix a bound violation, the system is `unsat`
- The choice of ordering $<$ and careful pivoting rules prevent cycling

28 Summary of the General Simplex Method for LRA

General Simplex Method for LRA

The general simplex method for Linear Real Arithmetic (LRA) solves systems of linear constraints by maintaining a tableau and iteratively pivoting to find a feasible solution.

- **Input**: A set of LRA constraints
- **Transformation**: Convert the constraints into the general form

  $$Ax = 0$$

  with bounds

  $$l_i \leq x_i \leq u_i$$

- **Algorithm**: Use the simplex algorithm to find a feasible solution.
  - Maintain a tableau with basic and non-basic variables
  - Non-basic variables are at their bounds
  - If a basic variable violates its bound, pivot to bring it back within the bounds
- **Output**:
  - If the system is satisfiable, return `sat`
  - If the system is unsatisfiable, return `unsat`

28.1 Example of Initial Tableau Construction

Let us consider a simple linear programming problem to illustrate the creation of the initial Simplex tableau.

Example 28.1 (Simple Linear Programming Problem)**. Problem**: Maximize the objective function Z

$$Z = 3x + 2y$$

Subject to the following constraints:

$$\begin{aligned} 2x + y &\leq 18 \\ 2x + 3y &\leq 42 \\ x \geq 0, \quad y &\geq 0 \end{aligned}$$

Step 1: Convert to Standard Form

To use the Simplex method, we first convert the inequalities into equalities by introducing slack variables. A slack variable represents the unused amount of a resource. We introduce slack variables s_1 and s_2 for the first and second constraints respectively.

The constraints become:

$$\begin{aligned} 2x + y + s_1 &= 18 \\ 2x + 3y + s_2 &= 42 \\ x \geq 0, \quad y \geq 0, \quad s_1 \geq 0, \quad s_2 &\geq 0 \end{aligned}$$

All variables, including the slack variables, must be non-negative:

$$x \geq 0, \quad y \geq 0, \quad s_1 \geq 0, \quad s_2 \geq 0$$

We also rewrite the objective function to be in the same form as the constraints:

$$Z = 3x + 2y \quad \Rightarrow \quad 3x + 2y - Z = 0$$

Step 2: Create the Initial Simplex Tableau

The initial Simplex tableau is a matrix that represents this system of linear equations. Each row corresponds to an equation and each column corresponds to a variable. Here is the initial Simplex tableau:

Base	Z	x	y	s_1	s_2	RHS
Z	1	-3	-2	0	0	0
s_1	0	2	1	1	0	18
s_2	0	2	3	0	1	42

Explanation of the Initial Tableau:

Rows:

- The first row corresponds to the objective function: $3x + 2y - Z = 0$. The coefficients of x and y are negative because we moved them to the left side of the equation.
- The second row corresponds to the first constraint: $2x + y + s_1 = 18$.
- The third row corresponds to the second constraint: $2x + 3y + s_2 = 42$.

Columns:

- The first column corresponds to the objective function value: Z.
- The second and third columns correspond to the problem variables: x and y.
- The fourth and fifth columns correspond to the slack variables: s_1 and s_2.

- The last column corresponds to the right-hand side constant terms of the equations: 18 and 42.

Basic and Non-Basic Variables:

- In the initial tableau, the slack variables (s_1 and s_2) and Z are the basic variables. A variable is basic if it has a column with a single 1 and all other entries in that column are 0. The labels on the left side of the tableau indicate the basic variables for that row.
- The non-basic variables are the problem variables: x and y, and are initially set to 0.

Initial Feasible Solution:

The initial tableau gives us an initial basic feasible solution: by setting the non-basic variables to 0, we can read the values of the basic variables directly from the RHS column.

$$Z = 0, \quad x = 0, \quad y = 0, \quad s_1 = 18, \quad s_2 = 42$$

This solution satisfies all the constraints and is therefore feasible. The simplex method will start from this initial solution and iteratively improve it by pivoting to find an optimal solution.

28.2 Example: Unsatisfiable System

Let us consider another example that demonstrates how the simplex method detects unsatisfiability.

Example 28.2 (Unsatisfiable Constraint System)**. Problem**: Consider the following system of constraints:

$$\begin{aligned} x + 2y &\geq 1 \\ 2x + y &\geq 1 \\ 2x + 2y &\leq 1 \end{aligned}$$

Step 1: Convert to Standard Form

We convert the inequalities into equalities by introducing slack variables. The constraints become:

$$\begin{aligned} x + 2y - s_1 &= 0 \quad \text{with } s_1 \geq 1 \\ 2x + y - s_2 &= 0 \quad \text{with } s_2 \geq 1 \\ 2x + 2y - s_3 &= 0 \quad \text{with } s_3 \leq 1 \end{aligned}$$

Variable Ordering: $x < y < s_1 < s_2 < s_3$

Step 2: Initial Tableau

The initial tableau in matrix form is:

$$\begin{bmatrix} 1 & 2 & -1 & 0 & 0 \\ 2 & 1 & 0 & -1 & 0 \\ 2 & 2 & 0 & 0 & -1 \end{bmatrix}$$

The sets of variables are:

- $\mathcal{N} = \{x, y\}$ (non-basic variables)
- $\mathcal{B} = \{s_1, s_2, s_3\}$ (basic variables)

The reduced tableau (showing only non-basic variables) is:

	x	y
s_1	1	2
s_2	2	1
s_3	2	2

Step 3: Initial Assignment

The initial assignment is:

$$\begin{aligned}\alpha_1(x) &= 0\\ \alpha_1(y) &= 0\\ \alpha_1(s_1) &= 0\\ \alpha_1(s_2) &= 0\\ \alpha_1(s_3) &= 0\end{aligned}$$

Step 4: First Pivot

We observe that $\alpha_1(s_1) = 0$ violates the constraint $s_1 \geq 1$. We need to increase $\alpha(s_1)$. The variable x is the smallest suitable non-basic variable (according to the ordering). We compute:

$$V = \frac{1-0}{1} = 1$$

After the pivot, the new assignment is:

$$\begin{aligned}\alpha_2(x) &= \alpha_1(x) + V = 0 + 1 = 1\\ \alpha_2(y) &= 0\\ \alpha_2(s_1) &= x + 2y = 1 + 0 = 1\\ \alpha_2(s_2) &= 2x + y = 2 + 0 = 2\\ \alpha_2(s_3) &= 2x + 2y = 2 + 0 = 2\end{aligned}$$

The system of equations becomes:

$$\begin{aligned}x &= s_1 - 2y\\ s_2 &= 2(s_1 - 2y) + y = 2s_1 - 3y\\ s_3 &= 2(s_1 - 2y) + 2y = 2s_1 - 2y\end{aligned}$$

The new sets are:

- $\mathcal{N} = \{s_1, y\}$ (non-basic variables)
- $\mathcal{B} = \{x, s_2, s_3\}$ (basic variables)

The new tableau is:

	s_1	y
x	1	-2
s_2	2	-3
s_3	2	-2

Step 5: Second Pivot

We observe that $\alpha_2(s_3) = 2$ violates the constraint $s_3 \leq 1$. We need to decrease $\alpha_2(s_3)$. The variable s_1 is not suitable because its coefficient in the constraint for s_3 is positive; to decrease

$\alpha_2(s_3)$, we would need to decrease $\alpha_2(s_1)$, but we cannot because $\alpha_2(s_1) = 1$ is equal to the lower bound for s_1. The variable y is suitable: we can increase y to decrease s_3 because y's coefficient is negative in the s_3 row. We compute:

$$V = \frac{1-2}{-2} = \frac{1}{2}$$

After the pivot, the new assignment is:

$$\begin{aligned}
\alpha_3(y) &= 0 + \frac{1}{2} = \frac{1}{2} \\
\alpha_3(s_1) &= \alpha_2(s_1) = 1 \\
\alpha_3(x) &= 1 - 2 \cdot \frac{1}{2} = 0 \\
\alpha_3(s_2) &= 2 - 3 \cdot \frac{1}{2} = \frac{1}{2} \\
\alpha_3(s_3) &= 2 - 2 \cdot \frac{1}{2} = 1
\end{aligned}$$

The system of equations becomes:

$$\begin{aligned}
2y = 2s_1 - s_3 \quad &\Rightarrow \quad y = s_1 - \frac{s_3}{2} \\
x = s_1 - 2\left(s_1 - \frac{s_3}{2}\right) &= s_1 - 2s_1 + s_3 = -s_1 + s_3 \\
s_2 = 2s_1 - 3\left(s_1 - \frac{s_3}{2}\right) &= 2s_1 - 3s_1 + \frac{3s_3}{2} = -s_1 + \frac{3s_3}{2}
\end{aligned}$$

The new sets are:

- $\mathcal{N} = \{s_1, s_3\}$ (non-basic variables)
- $\mathcal{B} = \{x, s_2, y\}$ (basic variables)

The new tableau is:

	s_1	s_3
x	-1	1
s_2	-1	$\frac{3}{2}$
y	1	$-\frac{1}{2}$

Step 6: Detection of Unsatisfiability

We observe that $\alpha_3(s_2) = \frac{1}{2}$ violates the constraint $s_2 \geq 1$. We need to increase $\alpha_3(s_2)$.

- The variable s_1 has a negative coefficient, so to increase s_2 we would need to decrease s_1, but we cannot because s_1 is equal to its lower bound.
- The variable s_3 has a positive coefficient, so we could increase s_3 to increase s_2, but that is not possible because s_3 is equal to its upper bound.

Since no independent variable is suitable to increase s_2, the system is unsatisfiable. The algorithm returns `unsat`.

Pivoting Rule

The **pivoting rule** is a rule to choose the suitable non-basic variable to pivot to when there is more than one candidate.
The pivoting rule consists of using a fixed ordering on variables known as **Bland's rule**, which avoids cycles in iterations by always selecting the smallest variable (according to the ordering) that is suitable for pivoting.